

# **HIRIX AI RESUME SCREENING SYSTEM**

## **PROJECT REPORT**

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD  
OF THE DEGREE OF

## **BACHELOR OF TECHNOLOGY**

(Computer Science and Engineering)



### **Submitted By:**

Amandeep Kaur (2203395)

### **Submitted To:**

Prof. Jaswant Singh

Prof. Palak Sood

Prof. PreetKamal Singh

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**GURU NANAK DEV ENGINEERING COLLEGE**

**LUDHIANA, 141006**

June, 2026

# **ABSTRACT**

The recruitment industry today faces a critical challenge: the overwhelming volume of job applications that human recruiters must evaluate manually. For a single open position, hiring teams may receive hundreds or even thousands of resumes, making it practically infeasible to assess each candidate thoroughly within a reasonable timeframe. This inefficiency leads to prolonged hiring cycles, inconsistent evaluation, and the inadvertent overlooking of qualified candidates.

The Hirix AI Resume Screening System addresses this challenge by introducing an intelligent, automated resume screening platform that leverages Artificial Intelligence, Natural Language Processing, and Machine Learning to streamline the recruitment pipeline. The system accepts resumes in PDF format, extracts textual content using PyMuPDF with Tesseract OCR fallback for scanned documents, and applies advanced NLP preprocessing techniques including tokenization, lemmatization, stopword removal, and abbreviation expansion.

The core of the system lies in its multi-dimensional scoring engine. Each resume is evaluated against a given job description across four independent dimensions: skills matching (with fuzzy string comparison using RapidFuzz), experience compatibility, education qualification alignment, and semantic similarity (using TF-IDF cosine similarity, Jaccard coefficient, and optionally BERT-based sentence embeddings). These component scores are combined using configurable weights to produce a composite relevance score.

Additionally, the system incorporates an ensemble Machine Learning pipeline consisting of XGBoost, Random Forest, and Logistic Regression models. When trained on historical hiring data, these models provide a supplementary ML-based score that is blended with rule-based scores to

improve prediction accuracy. A two-stage pipeline architecture further optimizes performance: Stage 1 applies lightweight coarse prefilter on raw text to eliminate obviously irrelevant resumes before Stage 2 performs computationally expensive NLP feature extraction and matching.

The frontend is built using Streamlit, offering recruiters an interactive dashboard with real-time processing progress, tabbed result views (Relevant, Non-Relevant, Full Ranked List, Analytics), Plotly powered visualizations, and multi-format export (CSV, Excel, JSON). The backend orchestration is managed through a Python command-line interface with comprehensive argument support.

## **ACKNOWLEDGEMENT**

We would like to express my sincere gratitude to everyone who contributed to the successful completion of my project, " Hirix Ai Resume Screening System."

We are deeply grateful to the Principal Dr. Sehijpal Singh, Guru Nanak Dev Engineering College (GNDEC), Ludhiana, for providing us the opportunity and facilities to undertake this project work. The constant guidance and encouragement received from Dr. Kiran Jyoti, Head of the Department, Computer Science & Engineering, Guru Nanak Dev Engineering College (GNDEC), Ludhiana, have been invaluable, and we acknowledge this support with sincere thanks.

We express our profound gratitude to our project guide, Sir Jasmeet Singh, Sir Kunwardeep Singh and Madam Navdeep kaur, for their expert counsel, constructive feedback, and patient guidance throughout the course of this work. Their insights greatly enhanced the quality and completeness of this project. Their expertise and mentorship were instrumental in shaping the direction and quality of our work.

Finally, we owe a debt of gratitude to our families for their unconditional love, patience, and encouragement, which sustained us throughout this academic journey

Amandeep Kaur

## LIST OF FIGURES

| Fig. No. | Figure Description                              | Page No. |
|----------|-------------------------------------------------|----------|
| 1.1      | System Overview                                 | 5        |
| 2.1      | Structure of Incremental SDLC Model             | 22       |
| 3.1      | High Level Architecture Diagram                 | 26       |
| 3.2      | Component Architecture Diagram                  | 27       |
| 3.3      | Data Flow Diagram                               | 29       |
| 3.4      | Two Stage Pipeline Flow Diagram                 | 31       |
| 3.5      | Sequence Diagram – Resume Screening Pipeline    | 33       |
| 3.6      | State Diagram – Resume Lifecycle                | 34       |
| 3.7      | Activity Diagram – Resume Upload and Processing | 36       |
| 3.8      | Deployment Architecture Diagram                 | 36       |
| 3.9      | UML Diagram – Hirix Screening System Class      | 38       |
| 5.1      | Home Page & Sidebar Configuration               | 76       |
| 5.2      | Job Description Input Interface                 | 77       |
| 5.3      | Resume Upload Interface                         | 78       |
| 5.4      | Processing and Real-Time Feedback               | 78       |
| 5.5      | Two-Stage Pipeline Summary Banner               | 79       |
| 5.6      | Candidate Ranking Dashboard                     | 80       |
| 5.7      | Resume Analysis Interface                       | 81       |

## LIST OF TABLES

| Table No. | Table Description                        | Page No. |
|-----------|------------------------------------------|----------|
| 4.1       | Test Cases for Text Extraction Module    | 67       |
| 4.2       | Test Cases for Feature Extraction Module | 68       |
| 4.3       | Test Cases for Job Description Processor | 68       |
| 4.4       | Test Cases for Matching Engine           | 69       |
| 4.5       | Test Cases for Ranking Module            | 70       |

# TABLE OF CONTENTS

| Contents                                                                           | Page No.  |
|------------------------------------------------------------------------------------|-----------|
| Abstract                                                                           | i         |
| Acknowledgement                                                                    | iii       |
| List of Figures                                                                    | iv        |
| List of Tables                                                                     | v         |
| Table of Contents                                                                  | vi        |
| <b>Chapter 1: Introduction</b>                                                     | <b>1</b>  |
| 1.1 Introduction to Project                                                        | 1         |
| 1.2 Project Category                                                               | 1         |
| 1.3 Problem Formulation                                                            | 2         |
| 1.4 Identification/Recognition of Need                                             | 3         |
| 1.5 Existing System                                                                | 4         |
| 1.6 Objectives of project                                                          | 4         |
| 1.7 Proposed System                                                                | 5         |
| 1.8 Unique features of the proposed system                                         | 7         |
| <b>Chapter 2: Requirement Analysis and System Specification</b>                    | <b>9</b>  |
| 2.1 Feasibility study                                                              | 9         |
| 2.2 Software Requirement Specification                                             | 11        |
| 2.3 SDLC model to be used                                                          | 16        |
| <b>Chapter 3: System Design</b>                                                    | <b>24</b> |
| 3.1 Design Approach                                                                | 24        |
| 3.2 Detail Design                                                                  | 25        |
| 3.3 User Interface Design                                                          | 39        |
| 3.4 Methodology                                                                    | 46        |
| <b>Chapter 4: Implementation and Testing</b>                                       | <b>49</b> |
| 4.1 Introduction to Languages, IDE's, Tools and Technologies used for Project work | 49        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 4.2 Algorithm/Pseudocode used                                     | 57        |
| 4.3 Testing Techniques                                            | 63        |
| 4.4 Test Cases designed                                           | 67        |
| <b>Chapter 5: Results and Discussion</b>                          | <b>71</b> |
| 5.1 User Interface Representation                                 | 71        |
| 5.2 Snapshots of system with brief detail of each and discussion. | 76        |
| <b>Chapter 6: Conclusion and Future Scope</b>                     | <b>82</b> |
| 6.1 Conclusion                                                    | 82        |
| 6.2 Future Scope                                                  | 83        |
| <b>References</b>                                                 | <b>85</b> |

# CHAPTER 1

## INTRODUCTION

### 1.1 Introduction of Project

Built in Python, Hirix is an AI-based resume-(CV-) screening system that automates the process of evaluating candidates. It takes in PDF resumes and a job description and uses a two-stage pipeline to filter and rank candidates. The first stage is called Coarse Filter and operates on the raw extracted text of resumes, performing a quick keyword and skills overlap check. This is a cheap operation that discards obviously irrelevant resumes before any expensive processing. The core engine is powered by several open-source libraries for extracting and processing text from PDFs (supporting both text and scanned PDFs), natural language processing, and similarity computation (scikit-learn, sentence-transformers, spaCy, NLTK, PyMuPDF, pdfplumber, Tesseract OCR, and more). An ensemble of machine learning models (Logistic Regression, Random Forest, XGBoost) is optionally used for more accurate relevance scoring. It can be used either via a command-line interface (app.py) for batch processing, or via an interactive Streamlit web dashboard (streamlit\_app.py), with visualizations powered by Plotly. Results, which include ranked candidate lists, and detailed breakdowns of skills match, experience fit, education alignment and more, are exported as JSON files for further analysis.

### 1.2 Project Category

1. **Application:** Web Application (Streamlit-based interactive dashboard with CLI support)
2. **Domain:** Artificial Intelligence & Natural Language Processing (AI/NLP)
3. **Basis:** Industry-Based (HR & Recruitment Automation)

### **1.3 Problem Formulation**

With hundreds to thousands of resumes in today competitive job market, organizations will sometimes only receive a handful for one opening. The process of screener each resume against the job requirements is very tedious, time-consuming, labor-work and prone to errors. A human recruiters spends an average of 6–8-seconds per resume while screening heads, resulting in inconsistency during the hiring process that can expose unconscious bias and disqualify suitable talent. With the growing number of applications, manual screening methods do not hold up anymore as they lead to a longer hiring cycle, higher recruitment costs and poor candidate-job alignment.

Moreover, resumes are present in different formats - text PDFs, scanners, image-based - preventing to extract information and represent it consistently as a standard approach. But job descriptions consists of explicit requirements (specific skills, qualifications and experience) as well explicit expectations that can be difficult even to quantify manually. The task then is to design a smart, automated system that can reliably extract structured information from unstructured resume documents, parse and understand job description requirements as well as benchmarks these against each other in multi-dimensional comparison - matching on skills, experience relevance and educational background to classify the candidates as relevant or non-relevant in the fastest manner possible with scale and confidence in scale (and no bias).

We propose a solution to tackle these challenges by framing the resume screening task as a multi-stage NLP & ML pipeline that takes multiple PDF resumes along with a job description as inputs and outputs a ranked, classified set of candidates, consequently automating the recruitment process initial screening phase and allowing HR teams to prioritise their efforts on the most suitable candidates.

## **1.4 Identification/Recognition of Need**

The traditional process of recruiting resumes was being observed, which triggered into the need of a system that can automatically screen resume. The most important thing to remember with recruiting these days is that the best job in the world for you has already been posted online, recruiters using easy-apply features are receiving an overwhelming number of applications - a staggering 250 to 1,000+ resumes per role. It is not only very time intensive but there are huge discrepancies as each resume could be looked at differently by different recruiters based on humans fatigue, personal bias or just another luck play or misinterpreted job requirement. It has been found that 75% of eligible job seekers are rejected at the screening stage when using manual processes due to human error or omission, which inherently causes a waste of talent.

Moreover, recruitment tools on the market are complex to integrate with the enterprise HR systems and can be expensive while offering no transparency on candidate scores or filtered selections. Most commercial ATS use straight logic keyword matching which does not understand contextual relevance — if the resume says "ML" it would be fail with only keywords as match, but anything beyond fragment matching is beyond most existing commercial systems. There is always a demand of a tool which can understand resumes in various format such as scanned pdf or image-based pdf where simple text-matching tools are unable to read the documents. Addressing these gaps led to the realization that we needed a smarter, transparent and cost-efficient resume screening solution which interprets resumes using Natural Language Processing and Machine Learning other than relying on keyword matching alone. The system must understand the semantic meaning of skills, be able to normalize experience and education contextually, support varying document formats using OCR, and display results in an easy-to-use interface — allowing small/mid-sized organizations and individuals to screen resumes quickly, accurately & with no bias.

## **1.5 Existing System**

Most of the existing resume screening systems rely on low-level manual recruitment processes and Applicant Tracking Systems (ATS). Manual screening is time-consuming, inconsistent with the evidence and not amenable to processing large volumes of applications. Commercial ATS platforms also automate screening but rely primarily on keyword matching and have a hard time understanding synonyms of keywords, context, or scanned resumes. These systems can be expensive and may not provide a full view of their ranking processes. Many of the open-source solutions that exist will either use TF-IDF and cosine similarity based techniques, but they generally do not evaluate an applicant for their skills, experience and education in a well-rounded manner nor provide a recruiter-friendly interface. Moreover, most existing methods use one year (or more) long filtering, where unnecessary computational complexity arises due to redundant filtering in-between. Such limitations necessitate the requirement of an intelligent, transparent and cost-effective solution which is offered by Hirix.

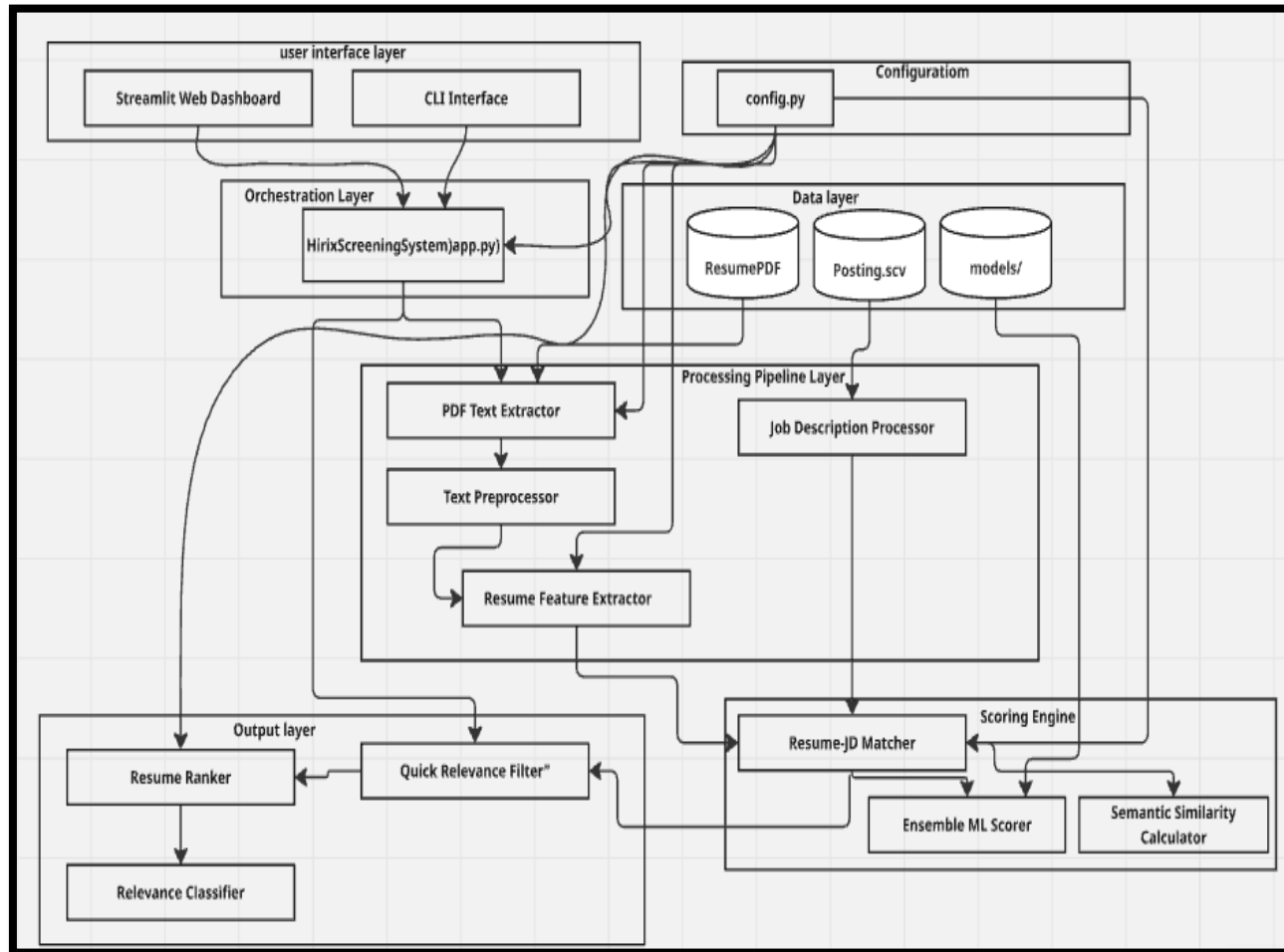
## **1.6 Objectives**

1. To develop an AI-powered resume screening system capable of extracting textual data from PDF resumes.
2. To automate the extraction of candidate attributes and perform intelligent ranking and classification based on job requirements.
3. To implement a two-stage resume screening architecture that combines coarse filtering and detailed evaluation for efficient candidate assessment.

## 1.7 Proposed System

### 1.7.1 System Overview

The proposed Hirix AI Resume Screening System:



*Fig. 1.1 System Overview*

### 1.7.2 Key Component

- **PDF Text Extraction Module:** Uses PyMuPDF (fitz) for primary text extraction with Tesseract OCR fallback for scanned documents. Automatically detects whether direct extraction yields sufficient text and switches to OCR when needed.

- **NLP Preprocessing Module:** Implements a multi-step pipeline comprising basic text cleaning, special character removal, whitespace normalization, tokenization (NLTK word\_tokenize), stopword removal (English + domain-specific), lemmatization (WordNet), POS-tag-based skill candidate extraction, and abbreviation expansion.
- **Feature Extraction Module:** Extracts 15+ categories of features from resume text using regex patterns, dictionary-based lookup (150+ technical skills, 25+ soft skills), and optionally spaCy NER. Features include contact information, technical skills, soft skills, years of experience, experience level, job titles, companies, degrees, fields of study, institutions, certifications, projects, and achievements.
- **Job Description Processor:** Parses free-text job descriptions to extract required and preferred skills, experience range, education requirements, responsibilities, location, remote work policy, salary range, and job type.
- **Multi-Dimensional Matching Engine:** Computes four independent scores — skills (40% weight), experience (25%), education (15%), and semantic similarity (20%) — using configurable weights. Skills matching incorporates fuzzy string comparison via RapidFuzz with a threshold of 80%.
- **Ensemble ML Pipeline:** Combines XGBoost Regressor, Random Forest Regressor, and Logistic Regression with weights 0.40, 0.35, and 0.25 respectively. Includes an optional stacking meta-model. When ML models are untrained, a heuristic fallback is employed.
- **Two-Stage Pipeline:** Stage 1 (Coarse Filter) applies lightweight keyword overlap and skills coverage checks on raw text to eliminate obviously irrelevant resumes. Stage 2 (Fine Pipeline) performs full NLP processing on passed resumes.

- **Ranking and Classification:** Sorts resumes by composite score and classifies them into four categories (Highly Relevant  $\geq 80\%$ , Relevant  $\geq 50\%$ , Low Relevance  $\geq 30\%$ , Not Relevant  $< 30\%$ ) with per-resume rejection reasons.
- **Interactive Dashboard:** Streamlit-based UI with sidebar configuration controls, three JD input methods, resume upload/dataset browsing, tabbed result views, Plotly charts, and multi-format export.

### 1.8 Unique features of the proposed system

**1. Two-Stage Filtering (Coarse + Fine):** Unlike existing systems that process every resume equally, Hirix first runs a quick, low-cost filter to remove obviously irrelevant resumes, and only then applies detailed NLP analysis on the rest. This makes the system significantly faster and more efficient.

**2. OCR-Based Resume Handling:** Most resume screening tools can only read text-based PDFs. Hirix can also read scanned documents and image-based PDFs using Tesseract OCR, ensuring no resume is skipped due to format issues.

**3. Multi-Dimensional Scoring:** Instead of giving a single keyword-match score, Hirix evaluates candidates on three separate dimensions — skills, experience, and education — giving recruiters a clearer and more detailed picture of each candidate's fit.

**4. Semantic Understanding:** The system uses sentence-transformers to understand meaning, not just keywords. For example, it recognizes that "ML" and "Machine Learning" refer to the same thing, which basic keyword-matching systems fail to do.

**5. Ensemble ML Scoring:** Hirix combines three different machine learning models (XGBoost, Random Forest, Logistic Regression) and takes their combined prediction, making the scoring more accurate and reliable than using any single model.

**6. Automatic Relevance Classification with Reasons:** Each resume is not just scored but also classified with a clear label (Highly Relevant, Relevant, Partially Relevant, Not Relevant) along with specific rejection reasons, so recruiters know exactly why a resume was filtered out.

**7. Dual Interface (Web + CLI):** The system offers both an interactive Streamlit dashboard for non-technical users and a command-line interface for developers or batch processing, making it versatile for different user types.

**8. Cost-Effective and Open-Source Stack:** Built entirely on free, open-source libraries (spaCy, NLTK, scikit-learn, XGBoost, Streamlit), Hirix requires no paid subscriptions or enterprise licenses, making it accessible to small teams and individual recruiters.

## CHAPTER 2

### REQUIEREMENT ANALYSIS AND SPECIFICATION

#### 2.1 Feasibility Study

The feasibility study evaluates technical, operational, and economic aspects of developing the HirIX AI-Powered Resume Screening System using Python, spaCy, NLTK, scikit-learn, sentence-transformers, XGBoost, PyMuPDF, Tesseract OCR, Streamlit, and Plotly.

##### 2.1.1 Technical Feasibility

- **Programming Language:** Python is used as the core language, which is widely supported and has extensive libraries for NLP and Machine Learning.
- **NLP Processing:** spaCy and NLTK provide robust text parsing, tokenization, and entity extraction capabilities.
- **Machine Learning Models:** scikit-learn and XGBoost offer well-tested algorithms for classification and relevance scoring.
- **Semantic Matching:** Sentence-Transformers enable contextual similarity matching using pre-trained deep learning models.
- **PDF Text Extraction:** PyMuPDF, pdfplumber, and Tesseract OCR ensure reliable extraction from text-based, scanned, and image-based PDFs.
- **Web Interface and Visualization:** Streamlit and Plotly provide a lightweight framework for building interactive dashboards with charts.
- **Open-Source Ecosystem:** All technologies are free, well-documented, and have active community support, making development technically viable.

**Conclusion:** All required technologies are mature, open source, well-documented, and available on the Python Package Index (PyPI). The technical requirements are fully satisfied.

### 2.1.2 Economic Feasibility

- **No Licensing Cost:** Built entirely on free and open-source libraries with no paid APIs or subscriptions.
- **Low Hardware Requirement:** Runs on standard PC/laptop without needing cloud GPUs or expensive infrastructure.
- **Reduced Recruitment Cost:** Automates initial screening, significantly reducing time and cost compared to manual screening or commercial ATS platforms.
- **Low Maintenance Cost:** Uses stable, widely adopted libraries with long-term community support, minimizing maintenance overhead.

**Conclusion:** The project incurs negligible development costs while delivering substantial time and cost savings, making it economically feasible.

### 2.1.3 Operational Feasibility

- **User-Friendly Interface:** Streamlit-based web dashboard requires no technical knowledge to operate.
- **Dual Access Mode:** Both web UI and CLI are available, catering to non-technical recruiters and technical users respectively.
- **Easy Deployment:** Runs on any machine with Python installed, requiring no complex server setup or third-party dependencies.
- **Result Export:** JSON-based output enables easy sharing, storage, and integration with existing HR workflows.

- **Minimal Training:** End users need minimal training to upload resumes, enter job descriptions, and interpret results.

**Conclusion:** The system is operationally feasible, with a low learning curve and high immediate value for target users.

## 2.2 Software Requirement Specification (SRS):

### 2.2.1. Data Requirement

- **Input Data:** The system requires candidate resumes in PDF format (text-based, scanned, or image-based) and job descriptions in plain text.
- **Job Postings Dataset:** A CSV file (postings.csv) containing job postings data for model training and reference.
- **Resume Storage:** Uploaded resumes are stored in a designated directory (uploads/temp\_uploads) during processing.
- **Trained ML Models:** Pre-trained ensemble model files (XGBoost, Random Forest, Logistic Regression) stored in the models/ directory in joblib format.
- **NLP Resources:** spaCy language model (en\_core\_web\_sm), NLTK corpora (stopwords, punkt, wordnet), and sentence-transformer models for semantic analysis.
- **Output Data:** Screening results are generated in structured JSON format containing scores, rankings, classifications, and analytics.
- **Log Data:** System activity and errors are recorded in a log file (hirix.log) for monitoring and debugging.

### 2.2.2. Functional Requirement

- **Resume Upload:** The system shall allow users to upload one or more PDF resumes through the web interface or specify a directory via CLI.
- **Job Description Input:** The system shall accept job descriptions as typed text, pasted content, or file input.
- **Text Extraction:** The system shall extract text from PDF resumes using PyMuPDF and pdfplumber, with Tesseract OCR as fallback for scanned documents.
- **Feature Extraction:** The system shall automatically extract skills, work experience, education, and certifications from resume text.
- **JD Parsing:** The system shall parse job descriptions into structured requirements including required skills, preferred skills, experience level, and education qualifications.
- **Coarse Filtering:** The system shall perform a quick keyword-based pre-filter to eliminate obviously irrelevant resumes before detailed processing.
- **Matching and Scoring:** The system shall calculate match scores across skills, experience, and education dimensions for each resume against the job description.
- **Semantic Matching:** The system shall optionally perform semantic similarity matching using sentence-transformers.
- **ML-Based Scoring:** The system shall optionally use ensemble ML models for relevance prediction.
- **Ranking:** The system shall rank all resumes by composite score in descending order.
- **Classification:** The system shall classify each resume as Highly Relevant, Relevant, Partially Relevant, or Not Relevant based on configurable thresholds.

- **Result Display:** The system shall display ranked results with scores, matched/missing skills, and rejection reasons on the dashboard.
- **Result Export:** The system shall export results in JSON format.

### 2.2.3. Performance Requirement

- **Processing Speed:** The system shall process at least 10–20 resumes per second during the coarse filtering stage.
- **Batch Capacity:** The system shall handle up to 1,000 resumes in a single screening session.
- **Response Time:** The web dashboard shall load and display results within 5 seconds after processing is complete.
- **Text Extraction Speed:** Each PDF text extraction shall complete within 2–3 seconds per document.
- **Memory Usage:** The system shall operate within 4 GB RAM for a batch of up to 500 resumes.
- **Scalability:** The two-stage pipeline shall ensure that processing time scales efficiently by skipping detailed analysis on irrelevant resumes.

### 2.2.4. Dependability Requirement

- **Fault Tolerance:** The system shall continue processing remaining resumes if text extraction fails for individual files, logging failures without crashing.
- **OCR Fallback:** If primary text extraction methods fail, the system shall automatically fall back to OCR-based extraction.
- **Error Logging:** All errors and warnings shall be recorded in the log file (hirix.log) with timestamps for diagnosis.

- **Data Integrity:** The system shall ensure that extracted features and scores are accurately mapped to the correct resume throughout the pipeline.
- **Consistent Results:** Given the same input resumes and job description, the system shall produce identical rankings and scores across repeated runs.
- **Graceful Degradation:** If semantic or ML models are unavailable, the system shall fall back to keyword-based matching without failure.

### 2.2.5. Maintainability Requirement

- **Modular Architecture:** The system is divided into independent, well-defined modules (text\_extractor, preprocessor, feature\_extractor, jd\_processor, matcher, ranker, ensemble\_models) for easy maintenance and updates.
- **Configurable Parameters:** All thresholds, paths, and settings are centralized in a single configuration file (config.py) for easy modification.
- **Code Documentation:** All modules and functions include docstrings and inline comments for developer readability.
- **Logging Support:** Comprehensive logging across all modules enables quick identification and resolution of issues.
- **Independent Model Training:** ML models can be retrained independently using train\_models.py without modifying the core pipeline.
- **Open-Source Dependencies:** All libraries used are actively maintained open-source projects with long-term support.
- **Test Scripts:** Dedicated test files (test\_pipeline.py, test\_quick.py, test\_fixes.py) are included for verifying system functionality after changes.

### 2.2.6. Security Requirement

- **Local Processing:** All resume data is processed locally on the host machine; no data is sent to external servers or cloud APIs.
- **Temporary File Cleanup:** Uploaded resumes stored in temp\_uploads are available only during the session and can be cleared after processing.
- **No User Authentication Data:** The system does not collect or store any personal login credentials or user account information.
- **File Type Validation:** Only PDF files are accepted for upload; other file types are rejected to prevent malicious file execution.
- **No External Network Dependency:** The core screening pipeline operates entirely offline after initial setup, eliminating risks of data interception.
- **Access Control:** The system runs on the local machine or local network, restricting access to authorized users only.

### 2.2.7. Look and Feel Requirement

- **Clean Dashboard Layout:** The Streamlit web interface provides a clean, organized layout with clearly labeled sections for input, processing, and results.
- **Interactive Visualizations:** Plotly-based charts display score distributions, category breakdowns, and skill match analysis in an interactive and visually appealing manner.
- **Color-Coded Classification:** Relevant and non-relevant resumes are visually distinguished using color-coded labels (e.g., green for Highly Relevant, red for Not Relevant).
- **Progress Indicators:** The system displays progress bars and status messages during resume processing to keep users informed.

- **Responsive Design:** The Streamlit interface adapts to different screen sizes and can be accessed through any modern web browser.
- **Readable Output:** Results are presented in a structured, tabular format with clear headings, scores, and matched/missing skill lists for easy interpretation.
- **Consistent Typography:** The interface uses consistent fonts, spacing, and formatting throughout for a professional appearance.

## 2.3 SDLC Model

SDLC is a structured process used by software organizations to plan, develop, test, deploy, maintain, and evolve software systems. It provides a disciplined approach to improving software quality and the overall development lifecycle by defining clear phases, roles, deliverables, and feedback loops.

### 2.3.1 Adopted Model: Incremental Model

The Incremental Model has been adopted for the development of the Hirix AI-Powered Resume Screening System. In this model, the system is designed, implemented, and tested in a series of increments, where each increment delivers a functional and tested module that adds new capability to the overall system. This model was adopted because the Hirix system is composed of multiple independent yet interconnected modules — text extraction, preprocessing, feature extraction, job description processing, matching, ranking, ML-based scoring, and the user interface. Each of these modules has a well-defined responsibility and can be developed, tested, and validated independently before being integrated into the main pipeline. The Incremental Model was preferred over other SDLC models for the following reasons: the project requirements were well understood at a high level but the implementation details of individual modules (such as OCR accuracy, ML model selection, and scoring thresholds) required experimentation and iterative refinement. The model provided the flexibility to incorporate improvements based on testing results at each stage.

### 2.3.2 Key tasks were organized into increment based on the project's functional requirements

- **Increment 1** – Text Extraction Module
  - Developed the PDF text extraction module using PyMuPDF, pdfplumber, and Tesseract OCR.
  - Tested with various PDF formats (text-based, scanned, image-based) to ensure reliable extraction.
- **Increment 2** – Preprocessing and Feature Extraction
  - Built the text preprocessing pipeline for cleaning and normalizing extracted text.
  - Developed the feature extraction engine to extract skills, experience, education, and certifications using spaCy and NLTK.
- **Increment 3** – Job Description Processing and Matching
  - Implemented the job description parser to extract structured requirements.
  - Developed the multi-dimensional matching and scoring system (skills, experience, education).
- **Increment 4** – Ranking and Classification
  - Built the ranking engine to sort candidates by composite score.
  - Added relevance classification with configurable thresholds and rejection reasons.
- **Increment 5** – Two-Stage Pipeline (Coarse Filter)
  - Integrated the Stage 1 coarse filter for quick elimination of irrelevant resumes before detailed processing.
  - Optimized the pipeline for speed and efficiency.
- **Increment 6** – ML Ensemble Models
  - Added the ensemble ML module (XGBoost, Random Forest, Logistic Regression).

- Developed the model training pipeline (train\_models.py) and integrated ML-based scoring.
- **Increment 7 – Semantic Similarity**
  - Integrated sentence-transformers for contextual matching beyond keyword overlap.
- **Increment 8 – User Interface and Result Export**
  - Developed the interactive Streamlit web dashboard with Plotly visualizations.
  - Added CLI support for batch processing and JSON-based result export.

### 2.3.3 Activities and Quality Gates

- **Project Setup and PDF Text Extraction**

Activities:

- Initialized project structure, virtual environment, and installed dependencies.
- Developed text extraction module with PyMuPDF, pdfplumber, and Tesseract OCR.

Quality Gate:

- Text must be successfully extracted from at least 95% of test PDF resumes including text-based, scanned, and image-based formats.
- OCR fallback must activate automatically when primary extraction returns empty or unreadable text.

- **Text Preprocessing and Feature Extraction**

Activities:

- Built text cleaning and normalization pipeline.
- Developed NLP-based extraction of skills, experience, education, and certifications.

Quality Gate:

- Preprocessed text must be free of noise, special characters, and irrelevant content.

- Feature extractor must correctly identify at least 80% of key skills and qualifications from sample resumes.

- **Job Description Processing**

Activities:

- Developed JD parser to extract required skills, preferred skills, experience level, and education qualifications.
- Tested across multiple job domains.

Quality Gate:

- Parser must correctly extract and categorize required and preferred skills from at least 90% of test job descriptions.
- Structured output must be consistent and complete across different JD formats.

- **Matching and Scoring Engine**

Activities:

- Built multi-dimensional matching across skills, experience, and education.
- Integrated optional semantic similarity using sentence-transformers.

Quality Gate:

- Matching scores must reflect accurate resume-JD alignment across all three dimensions.
- Semantic matching must correctly identify contextual synonyms (e.g., "ML" = "Machine Learning").
- No resume must receive a score without being evaluated on all applicable dimensions.

- **Ranking and Classification**

Activities:

- Developed ranking engine and relevance classification with configurable thresholds.

- Added rejection reason tracking for non-relevant resumes.

Quality Gate:

- Rankings must be in strictly descending order of composite score.
- Classification labels must correctly correspond to the configured threshold values.
- Every non-relevant resume must have at least one documented rejection reason.

- **Two-Stage Pipeline**

Activities:

- Implemented Stage 1 coarse filter for quick pre-screening.
- Integrated with Stage 2 fine pipeline.

Quality Gate:

- Coarse filter must not reject any resume that scores above the relevance threshold in the fine pipeline (no false negatives above tolerance).
- Stage 1 must reduce Stage 2 processing load by at least 30% on irrelevant-heavy datasets.
- Pipeline must execute end-to-end without errors.

- **Ensemble ML Models**

Activities:

- Developed XGBoost, Random Forest, and Logistic Regression models.
- Built training pipeline and integrated ML scoring.

Quality Gate:

- Trained models must achieve minimum acceptable accuracy on validation data.
- Ensemble prediction must produce consistent and reproducible scores across repeated runs.

- System must gracefully fall back to keyword-based scoring if ML models are unavailable.

- **User Interface and Result Export**

Activities:

- Developed Streamlit web dashboard with Plotly visualizations.
- Implemented CLI mode and JSON result export.

Quality Gate:

- Dashboard must load without errors and display all sections — upload, input, results, and visualizations.
- File upload must accept only PDF format and reject invalid files.
- JSON output must be valid, well-structured, and contain all scored and classified resume data.
- CLI must execute with all configurable parameters without failure.

- **Testing and Optimization**

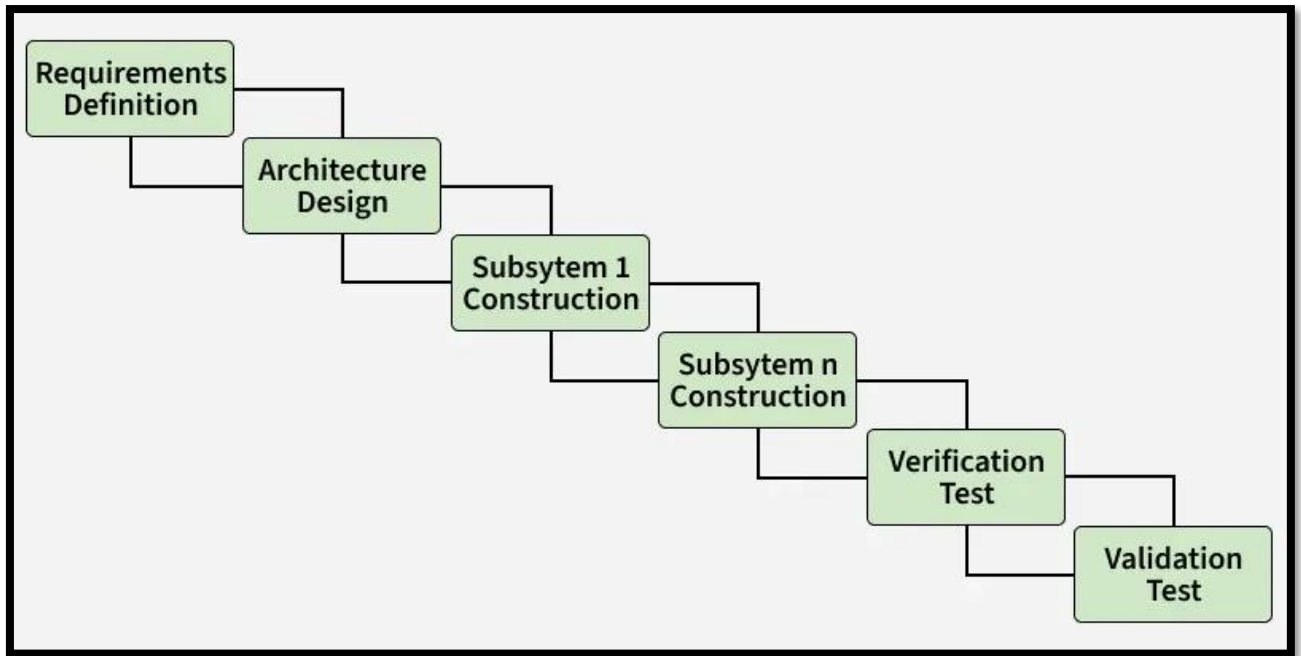
Activities:

- Conducted end-to-end testing with multiple job descriptions and resume sets.
- Performed performance tuning and bug fixing.

Quality Gate:

- All test scripts (test\_pipeline.py, test\_quick.py, test\_fixes.py) must pass without errors.
- System must handle a batch of 500+ resumes without crashes or memory overflow.
- Final output must match expected rankings for known test cases.
- All identified bugs must be resolved and verified before project completion.

### 2.3.4 Structure of Incremental SDLC Model:



*Fig. 2.1 Structure of Incremental SDLC Model*

#### **Phase 1 – Requirement Analysis (One-Time)**

- All system requirements were identified upfront — text extraction, preprocessing, feature extraction, JD processing, matching, ranking, ML scoring, and UI.
- Requirements were divided into increments based on module dependency and priority.

#### **Phase 2 – Increment Planning**

- Each increment was planned to deliver one or more independent, functional modules.
- Dependencies between modules determined the increment order (e.g., text extraction before feature extraction).

#### **Phase 3 – Design, Code, Test (Per Increment)**

- Each increment followed its own mini design → development → testing cycle.

- Modules were tested independently using dedicated test scripts before integration.

#### **Phase 4 – Integration and Delivery (Per Increment)**

- After testing, each new module was integrated into the existing system.
- A working version of the system was available after every increment.

# CHAPTER 3

## SYSTEM DESIGN

### 3.1 Design Approach

The Hirix AI Resume Screening System has been designed following the principles of modular architecture and separation of concerns. The system is decomposed into ten distinct Python modules, each responsible for a well-defined aspect of the processing pipeline. This modular approach offers several advantages:

- **Independent Development and Testing:** Each module can be developed, tested, and debugged independently, reducing the complexity of the development process.
- **Reusability:** Modules such as the TextPreprocessor and PDFTextExtractor are general-purpose components that can be reused in other NLP and document processing projects.
- **Maintainability:** Bug fixes and feature enhancements can be localized to the affected module without impacting the rest of the system.
- **Extensibility:** New scoring dimensions, ML models, or output formats can be added by introducing new modules or extending existing ones, without modifying the core pipeline.
- **Testability:** Each module exposes well-defined interfaces (classes and functions with typed parameters), enabling structured unit testing.

The design follows a pipeline architecture pattern, where data flows sequentially through a series of processing stages: Text Extraction → Preprocessing → Feature Extraction → Matching → Ranking → Classification → Visualization. Each stage transforms its input and passes the result to the subsequent stage. The system also employs the Strategy Pattern for scoring, allowing different scoring strategies (rule-based, ML-based, or hybrid) to be selected at runtime through configuration.

## 3.2 Detail Design

The conceptual design is expanded upon in the Detailed Design of the Hirix AI Resume Screening System by taking system requirements and decomposing them into a precise technical specification. It describes the architecture, data flow and how components interact for smart automated resume screening. So the system is made of a modular multi-layered architecture in which the User Interface (Streamlit Dashboard and CLI) connects with an Orchestrator that calls and coordinates two stages pipeline — coarse (text-level) pre-filtering on raw text followed by fine (sentence-level) NLP/ML based scoring.

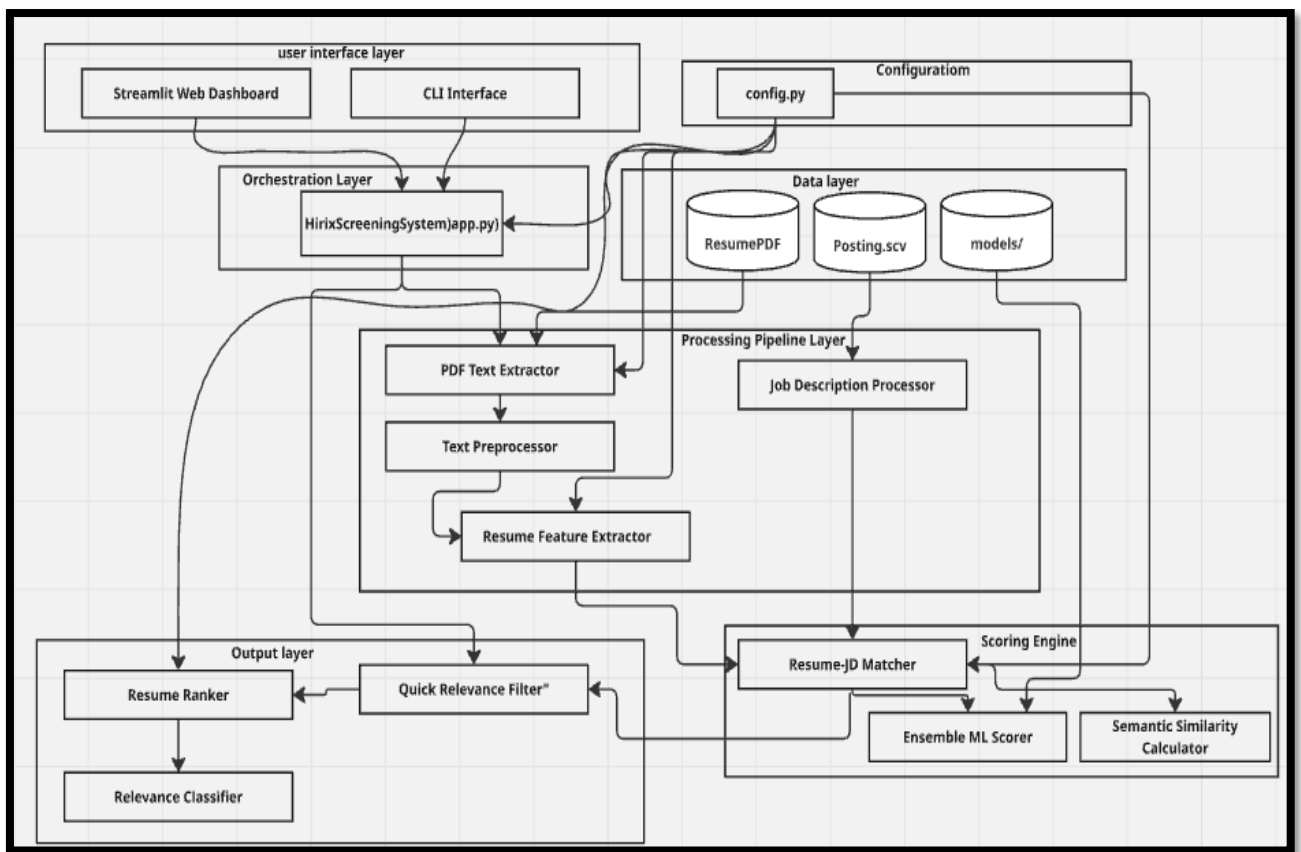
The architecture consists of flowcharts of the PDF text extraction, two-step screening pipeline, resume–JD matching and relevance classification. Various UML diagrams including use case diagrams, sequence diagrams and class diagram have been created to enlighten recruiter interactions, message flow between the modules and entity structures like ResumeFeatures, JobRequirements and RankedResume.

These structured designs are just what system needs for modularity, scalability, maintainability and upgradeability — specs are the key for AI application performance ( or performance of any an App in general) of resume screening application.

### 3.2.1. High-Level Architecture

Diagram 7 Layered blocks-T2H Flow of Hirix System At the top there are two entry points that connect into an orchestrator we call HirixScreeningSystem, this orchestrates the whole pipeline. When sending through the Processing Pipeline (text extraction → preprocessing → feature extraction), the orchestrator also parses the jd. The Matcher integrates rule-based matching, ML

ensemble scoring and semantic similarity to derive a final score together with the two outputs which feed directly into the Scoring Engine. The Output Layer sorts and categorizes resumes as either relevant/non-relevant along with a rough pre-filter that lets irrelevant resumes skip away at the earliest stage. config. py is connected through dotted lines (weights, thresholds, paths) to almost every module on the schema below while the Data Layer at the bottom holds PDF resumes, job postings CSV and trained ML models.

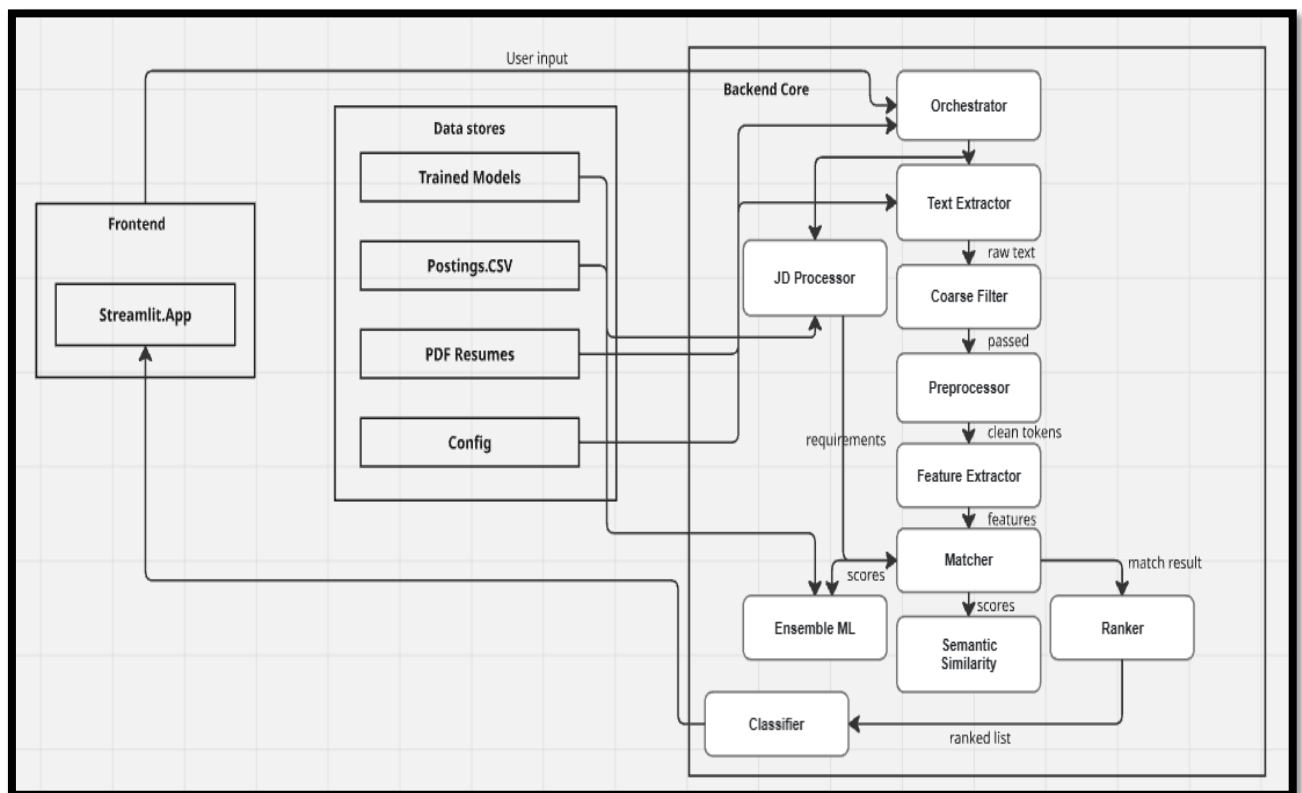


**Fig. 3.1 High-Level Architecture**

### 3.2.2 Component Architecture diagram

The Component Architecture Diagram is a left-to-right (LR) graph that illustrates the data flows via arrows with labels to show which in of three major sections contribute to, the Hirix system: Frontend,

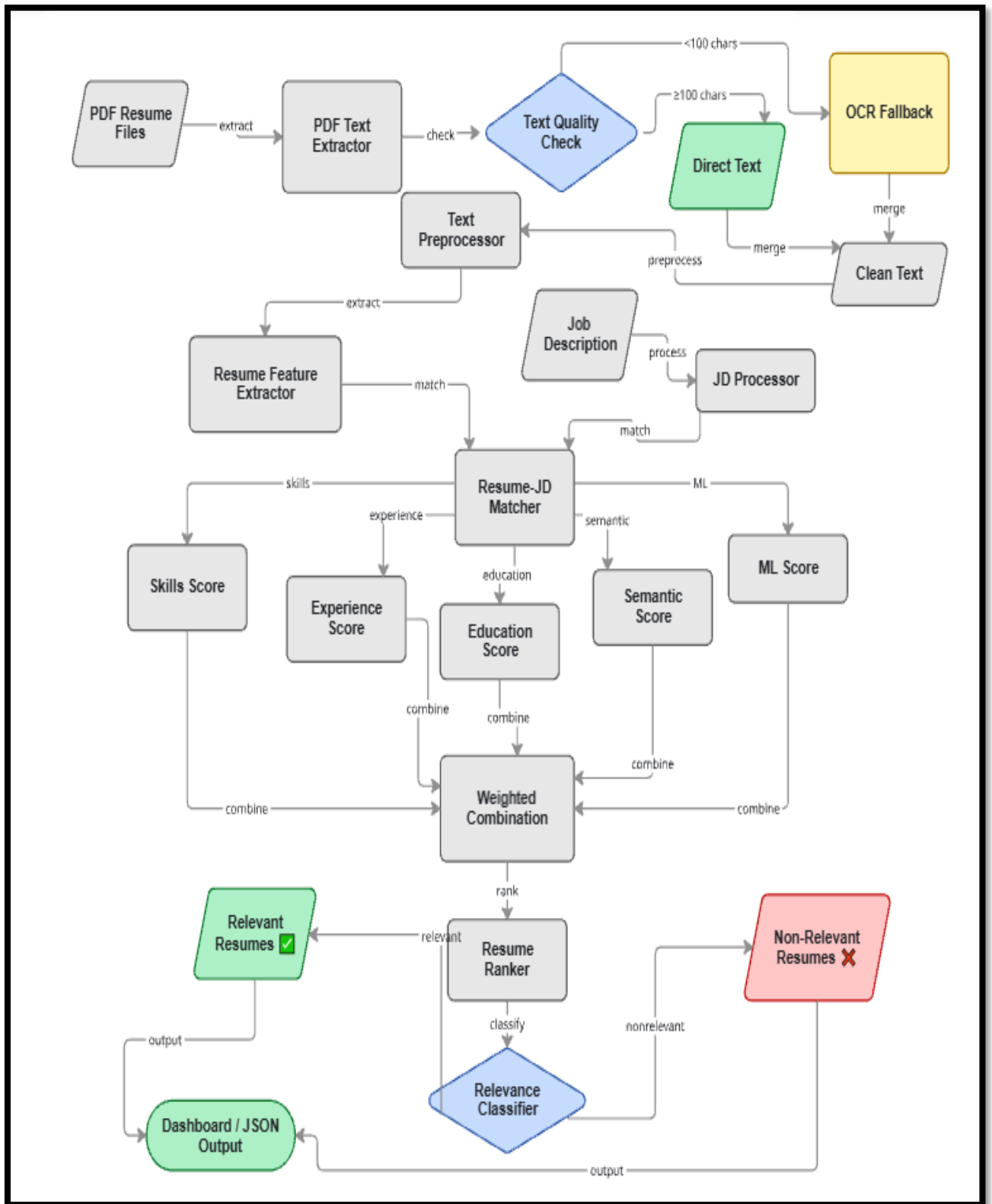
Backend Core, and Data Stores. The Frontend (Streamlit App) takes user input and forwards it to the Backend Core Orchestrator, which passes it via a sequential chain — Text Extractor→Raw text from PDFs→Coarse Filter→Preprocessor (for cleaningText)→Feature Extractor (structured data). Simultaneously, the JD Processor pulls job requirements from a job description. Matcher – feeds via features and requirements to the Matcher, which will be scored via Ensemble ML + Semantic similarity module. Results are then sorted by the Ranker and classified as relevant or non-relevant by the Classifier which in turn returns this final classified output to the Frontend. Data Stores (PDF Resumes, Postings CSV, Trained Models, Config) — These are the DataTrans of our components that provide the input data to their associated component. The below diagram simply gives you a horizontal view of how data flows componentto-component in the complete system.



**Fig. 3.2 Component Architecture diagram**

### 3.2.3 Data Flow Diagram

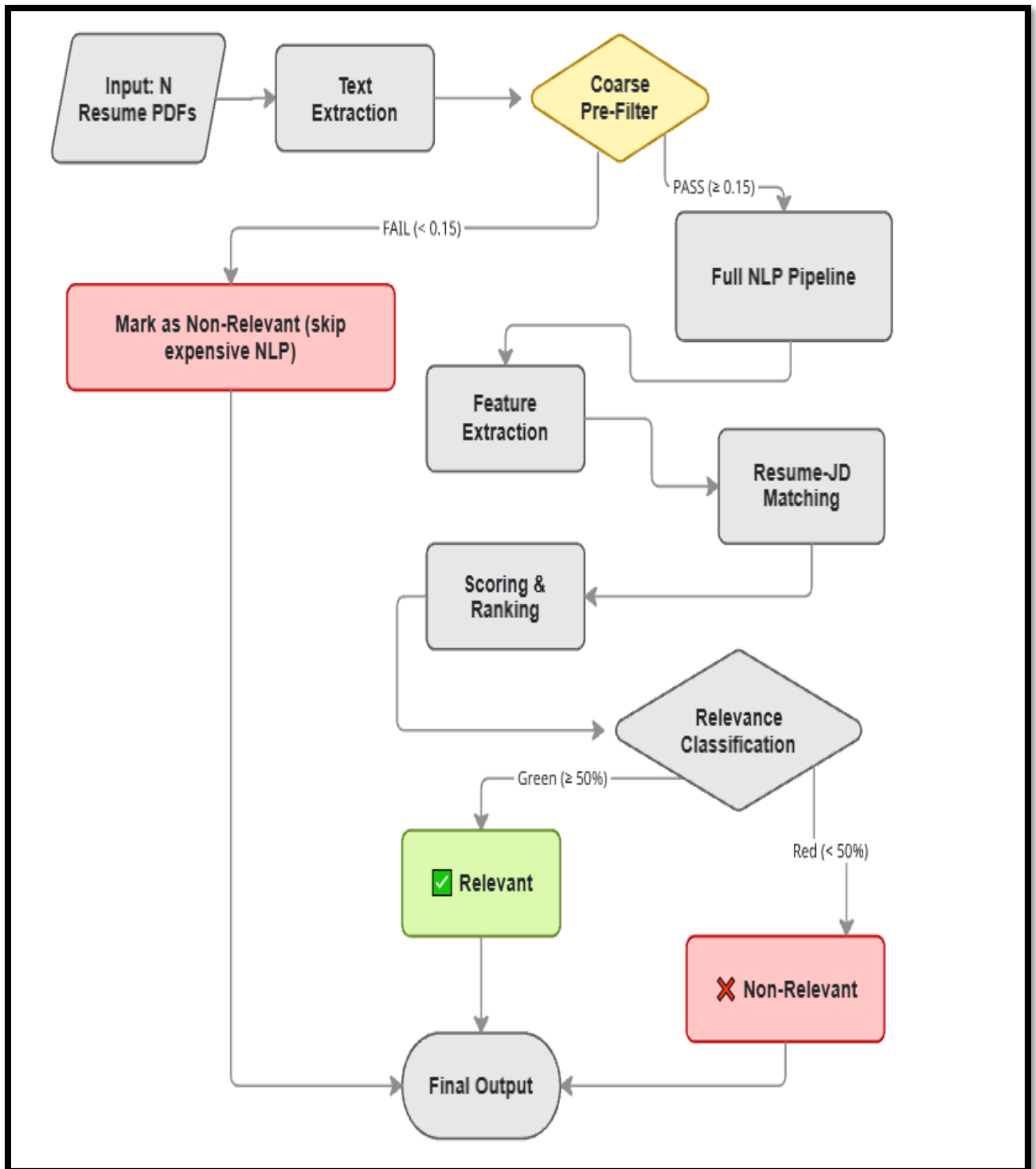
The Data Flow Diagram is a flowchart structured top to bottom showing how data transforms at every point in the pipeline of Hirix. PDF Resume Files pass into the PDFTextExtractor where it encounters a decision diamond, if the extracted text had  $\geq 100$  characters then it proceeded on direct path while less than that redirected to use OCR (Tesseract). And finally, it was gradually morphed into ranked, classified candidate results. These two paths combine as clean text, moving down the TextPreprocessor and ResumeFeatureExtractor pipelines to complete semi-structured resume features. Simultaneously, in a branch, the Job Description goes into the right-side pipe called JobDescriptionProcessor  $\rightarrow$  processes it and generates structured requirements. These two streams flow into the ResumeJDMatcher which produces five different scores separately - Skills Score, Experience Score, Education Score, Semantic score and ML Score. The five scores are given as inputs to a Weighted Combination block whose output is a total\_score, and the output of this weighted combination is passed to both the ResumeRanker for ranking and RelevanceClassifier for classification. The classifier segments results into Relevant and Non-Relevant, both of which contribute to the final dashboard/JSON output. This diagram provides a high-level overview of the transformation steps for extracting candidate results, from raw PDF files and job text to ranked (and classified) candidates.



**Fig.3.3 Data Flow Diagram**

### 3.2.4 Two Stage Pipeline flow

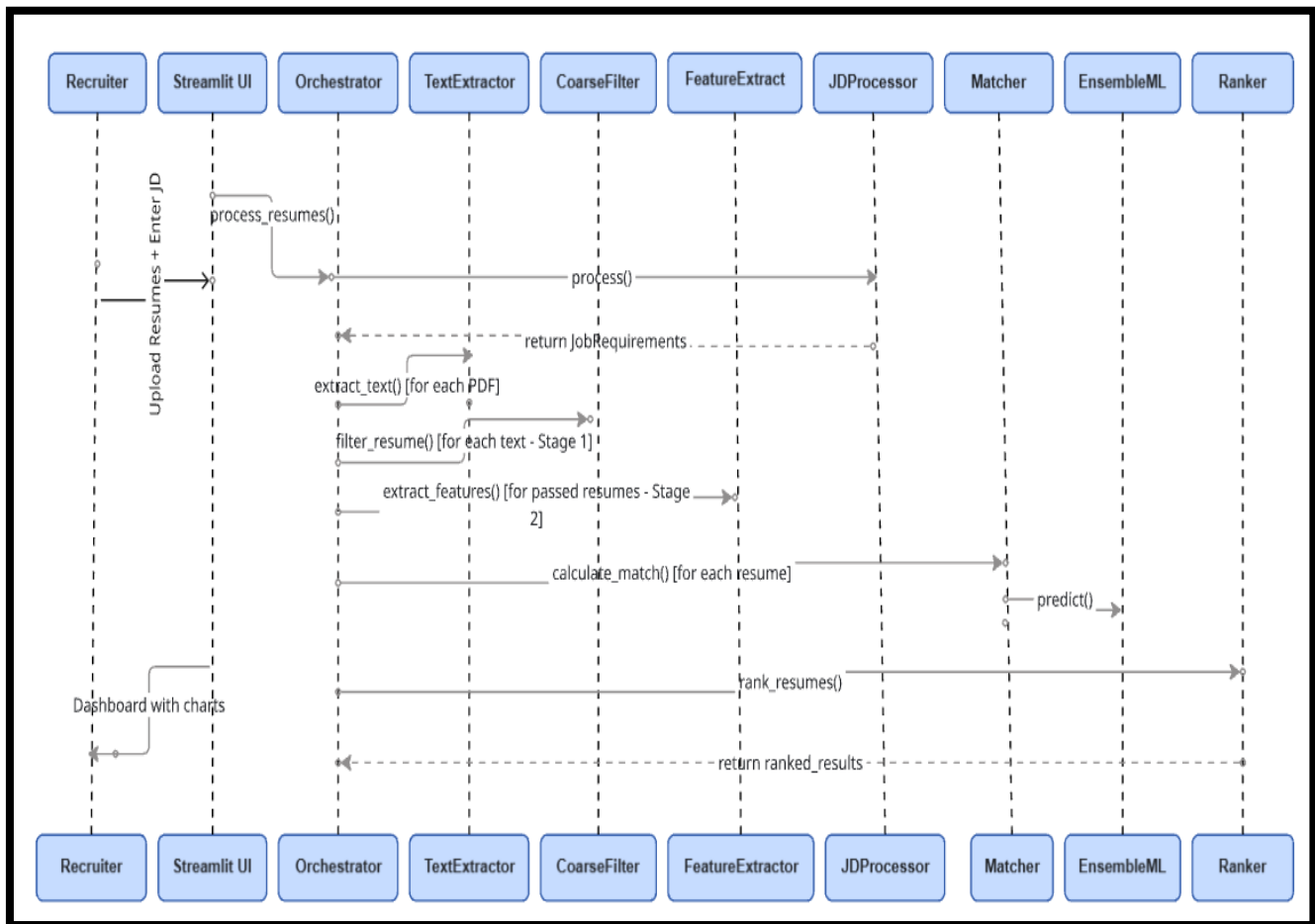
Hirix implements two-stage pipeline and the Two-Stage Pipeline Flow Diagram is used to better understand its efficiency of operation with the screening resumes in a two-stage fashion. From there it moves through N Resume PDFs arriving at Stage 0 (Text Extraction) and then reaches a yellow decision diamond — Stage 1: Coarse Pre-Filter. Resumes with  $\text{coarse\_score} \geq 0.15$  PASS and go to Stage 2 (Full NLP Pipeline), while if the score is less than 0.15 it FAILS immediately - next, in a red box we mark them as Non-Relevant skipping expensive processing all together using NLP pipeline. This leads to a second decision diamond Relevance Classification - the passed resumes then flow through Feature Extraction → Resume-JD Matching → Scoring & Ranking. In this case, resumes with scoring 50% and above = Relevance, below 50% – Non-Relevant. The Final Output is the preservation of all 3 paths (coarse-filtered rejects, fine-grained rejects, relevant resumes). The value of this diagram lies in displaying how the coarse filter saves a great deal of processing time by removing obviously unqualified resumes upfront, before they can even run through the more-DNN / computationally expensive NLP and ML scoring pipeline.



**Fig. 3.4 Two Stage Pipeline flow**

### 3.2.5 Sequence Diagram – Resume Screening Pipeline

This is a UML sequence diagram of the entire resume screening process as a message exchange sequence in time for 10 actors = Recruiter, Streamlit UI, Orchestrator, TextExtractor:Extract 400~500 resumes CoarseFilter:\* Pre-process resumesFeatureExtractor:Extract features and train145 models — Extracted from String Extractor using Coarse FilterJDProcessor:\* Process JD Matcher Inference EnsembleML (Ensemble Learning) Ranker: Fitting process - Comparing with company provided CV variants you can see that output data accumulated at various nodes. The process starts with the Recruiter passing in resumes and job description via UI that calls process\_resumes() on Orchestrator. The Orchestrator then performs 5 sequential steps:Step 1 — Send the job text to JDProcessor, which returns structured JobRequirements. Stage 2 — This one simply loops over the resumes that pass the filters and calling FeatureExtractor to extract skills, experience & education. Finally, step 4 iterates through every resume with Matcher (which hits EnsembleML for ML predictions) and return match results. Lastly,Step 5 returns all match results to the Ranker which sends it back through the UI to the Recruiter as analytical dashboard with charts. The diagram demonstrates the exact order of operations and depicts how each module communicates within a single screening session.

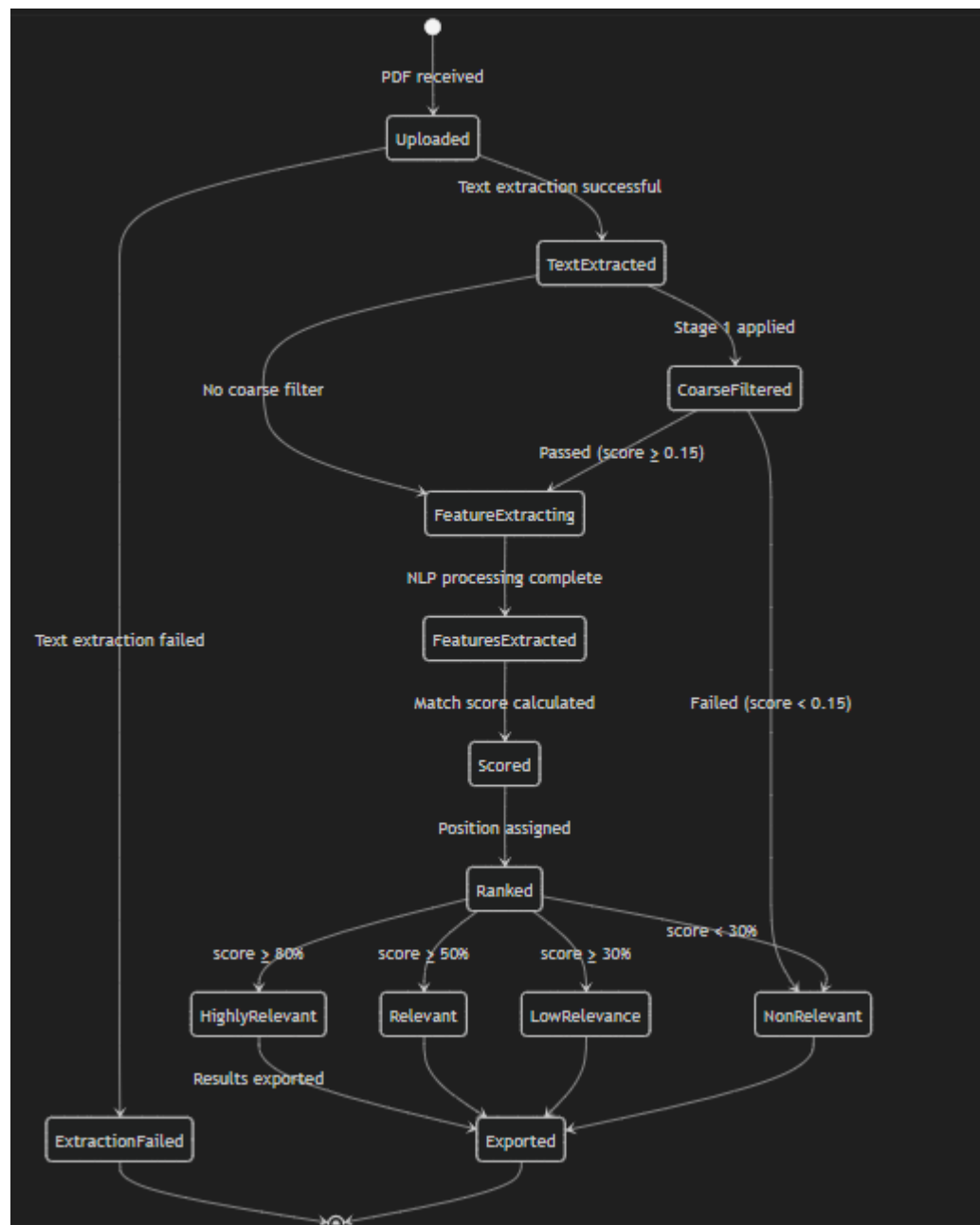


**Fig. 3.5 Sequence Diagram – Resume Screening Pipeline**

### 3.2.6 State Diagram – Resume Lifecycle

This is the state diagram showing the life cycle of a given resume within Hirix. On receiving a PDF it enters an Uploaded state and goes on to extract text. If extracting fails, the resume proceeds to ExtractionFailed. If the extraction is successful, the resume goes through TextExtracted and maybe passes through CoarseFiltered (where a rapid keyword pass drops blatantly bad resumes; score < 0.15) to reduce processing time Resumes that get through them go on to FeatureExtracting (NLP processing), Scored (match score calculation) and then Ranked (position assignment). Finally, each

resume is classified as HighlyRelevant ( $\geq 80\%$ ), Relevant ( $\geq 50\%$ ), LowRelevance ( $\geq 30\%$ ), or NonRelevant ( $< 30\%$ ) based on its score, and all results are Exported for download.



**Fig.3.6 State Diagram – Resume Lifecycle**

### **3.2.7 Activity Diagram - Resume Upload and Processing**

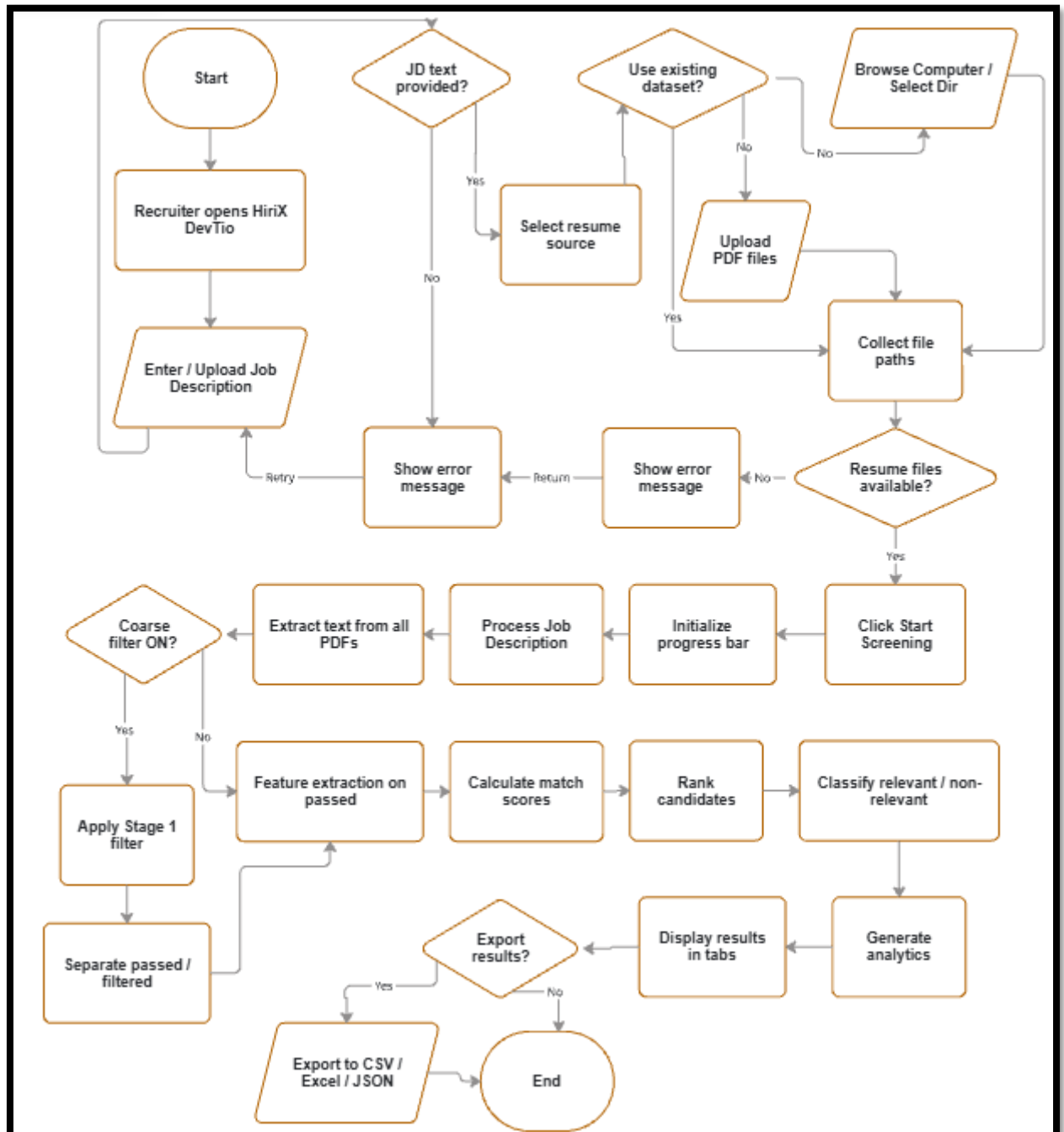
The Resume Upload and Processing Workflow in the Hirix AI Resume Screening System begins when a recruiter accesses the Streamlit-based dashboard and provides a job description, either by entering it manually or uploading it. The system first validates the job description to ensure that the required information has been provided. If the job description is missing or invalid, an appropriate error message is displayed, and the recruiter is prompted to correct the input before proceeding. Once a valid job description is accepted, the recruiter uploads candidate resumes in PDF format. The system performs validation checks on the uploaded files to verify the file type and ensure that the files meet the specified size requirements. Any invalid files are rejected, and the user is notified accordingly.

After successful validation, the resume screening process is initiated. The system executes its two-stage processing pipeline, which includes coarse pre-filtering of resumes, text extraction from PDF documents, NLP-based feature extraction, multi-dimensional matching against the job requirements, and final ranking with relevance classification. This automated workflow efficiently evaluates each candidate's suitability for the specified role.

Upon completion of the screening process, the results are presented through an interactive dashboard. Recruiters can review categorized outcomes using dedicated tabs for relevant resumes, non-relevant resumes, and visual analytics that provide insights into candidate performance and matching scores.

Finally, the system offers export functionality, allowing recruiters to download the screening results in CSV, Excel, or JSON formats for further analysis or record-keeping. The workflow illustrates the complete user interaction cycle, including validation checkpoints, decision-making stages,

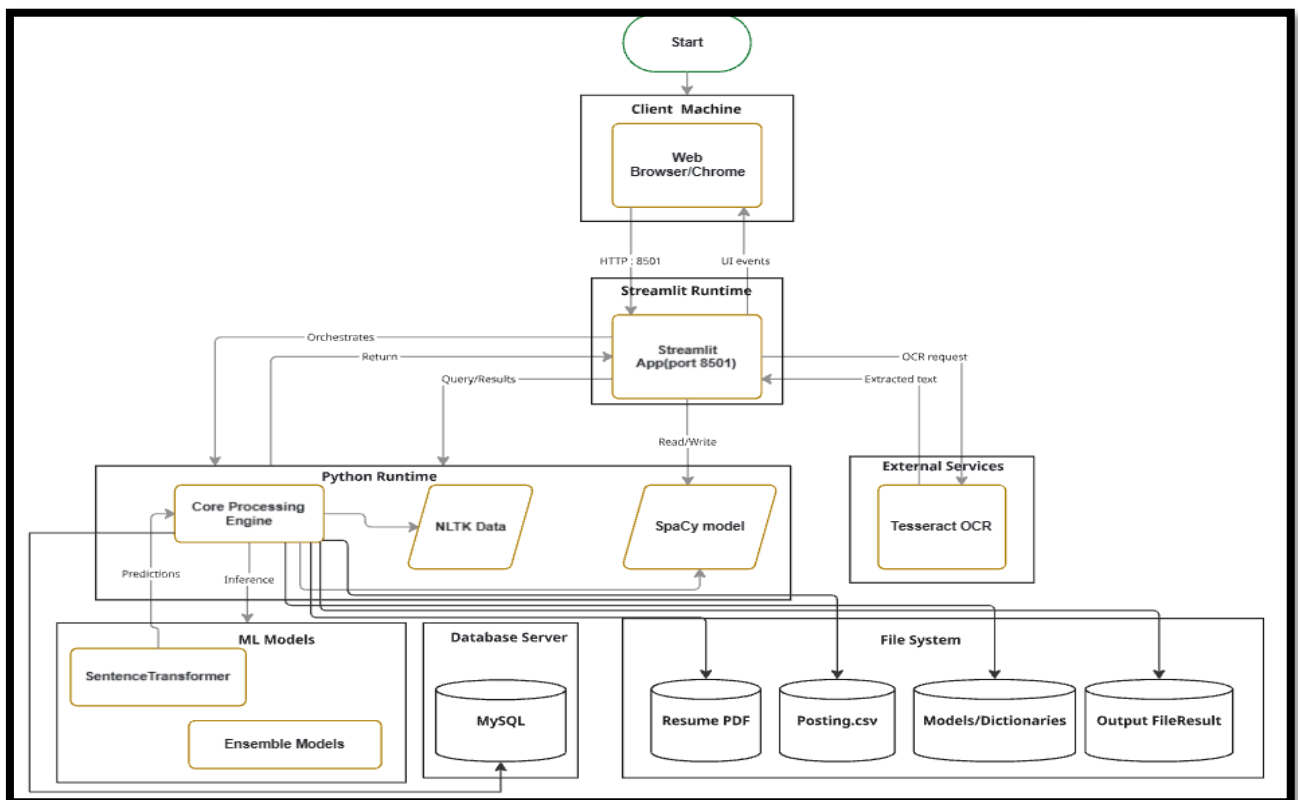
processing activities, and error-handling mechanisms that ensure a reliable and user-friendly screening experience.



**Fig.3.7 Activity Diagram - Resume Upload and Processing**

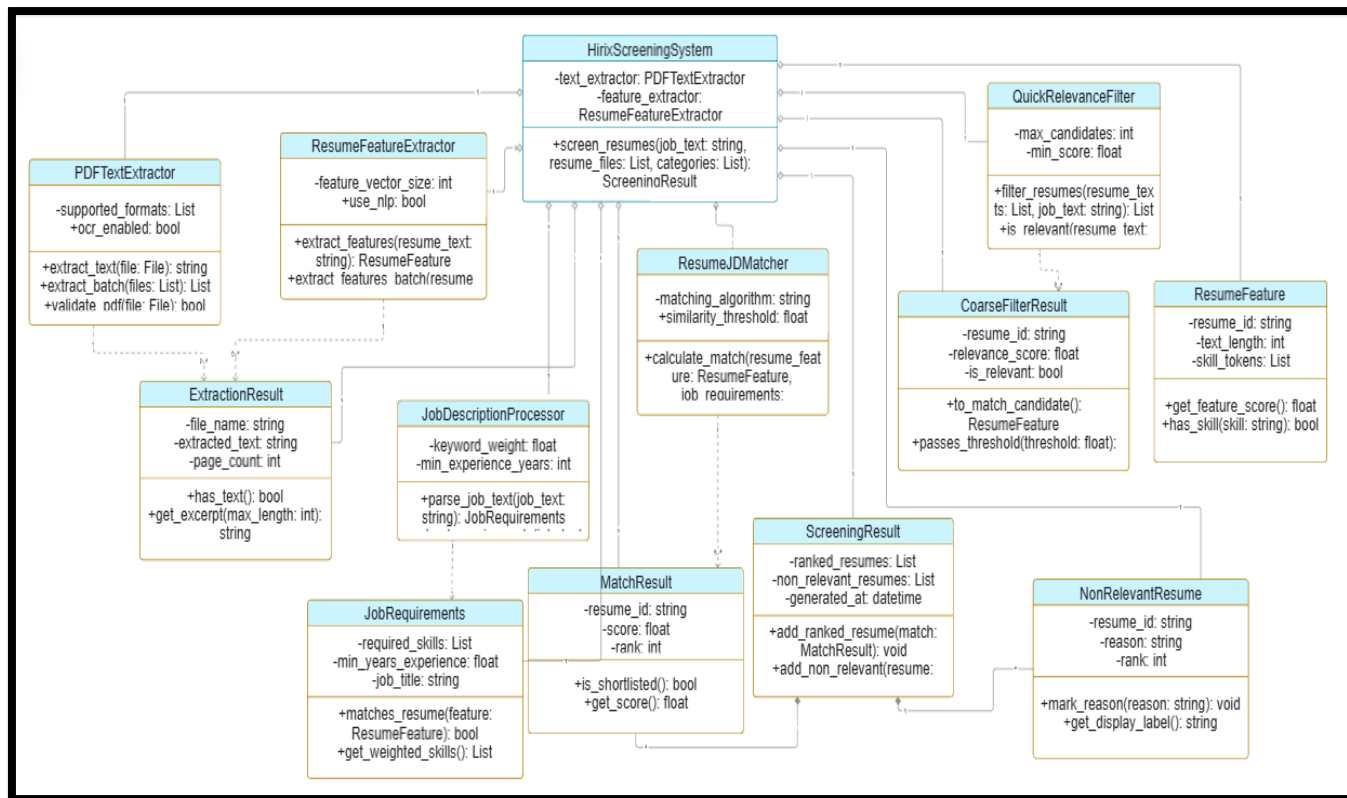
### 3.2.8 Deployment Diagram

The Deployment Architecture of the project illustrates how its software components are organized and interact within the system. Recruiters access the application through a web browser, which communicates with the Streamlit-based Application Server over HTTP. The server manages the Core Processing Pipeline and Machine Learning Models responsible for resume analysis, matching, and ranking. The system utilizes Tesseract OCR for text extraction from scanned resumes and spaCy with NLTK for natural language processing tasks. Resume files, datasets, configuration files, and trained models are stored in the local file system. A MySQL database is also included as a future enhancement for persistent storage of users, job descriptions, and screening results. The architecture highlights the system's current local deployment and its readiness for future cloud integration.



**Fig.3.8 Deployment Architecture**

### 3.3.9 UML Diagram - Hirix Screening System Class



**Fig.3.9 UML Diagram - Hirix Screening System Class**

### 3.3 User Interface Design

The user interface of the Hirix AI Resume Screening System is built using Streamlit, a Python-based framework for developing interactive web applications. It provides recruiters with a simple, single-page dashboard to input job descriptions, upload candidate resumes, configure screening parameters, and view AI-generated ranking results — all accessible through a standard web browser without requiring any technical expertise in AI or NLP.

#### 3.3.1 Design Objectives

The user interface of the Hirix AI Resume Screening System is designed to provide a simple, efficient, and professional experience for recruiters. Since the system is intended for HR professionals and non-technical users, the design focuses on clarity, ease of use, and smooth workflow execution. The interface ensures that the complete resume screening process can be performed with minimal effort while maintaining high usability and performance.

- **Simple and user-friendly interface:** Designed for non-technical users with clean layout, clear labels, and intuitive controls to ensure ease of operation.
- **Fast resume screening workflow:** Reduces multiple steps into a single streamlined process, enabling quick job description input, resume upload, and processing.
- **Easy navigation:** Provides structured layout with sidebar and main content area for smooth movement across different system functions.
- **Professional recruiter dashboard:** Displays results using ranked lists, score cards, and visual analytics to support data-driven decision making.
- **Clean and responsive design:** Maintains consistent styling, proper spacing, and responsive layout for better usability across different devices.

### 3.3.2 User Interface Architecture

The Hirix system follows a Streamlit-based single-page application (SPA) architecture where all components are rendered dynamically on a single interface without page reloads.

The system workflow is:

- Recruiter
- Job Description Input
- Resume Upload
- AI Processing
- Candidate Ranking
- Export Results

The interface is divided into:

- **Header Region:** Displays system title and branding
- **Sidebar Region:** Contains configuration controls and settings
- **Main Content Region:** Handles complete screening workflow
- Streamlit's reactive model ensures real-time updates on every interaction
- Session state is used to store uploaded files and processing results

### 3.3.3 Home Page Interface

The home page is the first screen presented to the recruiter upon launching the application in the web browser. It serves as the entry point to the screening workflow and provides the following elements:

- **Project Title:** The application title "Hirix AI Resume Screening" is displayed prominently at the top of the page using large, bold styling with a professional color theme and an accompanying icon for identification.
- **System Description:** A short subtitle explains the purpose of the system as an AI-powered resume screening and ranking tool for recruiters.
- **Sidebar Navigation:** The sidebar contains grouped configuration settings such as scoring thresholds, ML model selection, pipeline options, and system statistics like resume dataset size.
- **User Instructions:** The layout naturally guides the recruiter step-by-step from job description input to resume upload and then to screening execution without requiring separate instructions.

### 3.3.4 Resume Upload Interface

The resume upload section provides two methods for supplying candidate resumes to the screening system:

- **Single/Multiple PDF Upload:** The recruiter can upload one or more resumes in PDF format using Streamlit's file uploader, which supports multiple file selection. Uploaded files are temporarily stored for processing. Alternatively, resumes can be selected from a categorized dataset (e.g., Data Science, Software Engineer, HR) using a checkbox-based selection system that automatically collects all resumes from chosen folders.
- **File Validation:** Each uploaded file is validated before processing. Validation includes file existence, size check, and PDF format integrity verification. Any corrupted or invalid files are automatically excluded from the processing pipeline and reported to the recruiter.

- **Upload Status Messages:** The interface displays real-time messages such as the number of resumes successfully uploaded or selected (e.g., “Selected 150 resumes”). If dataset paths are missing or invalid, an error message is shown to guide corrective action.

### 3.3.5 Job Description Input Interface

The job description input section allows recruiters to define job requirements using multiple flexible methods:

- **JD Text Area Input:** A large text box is provided where recruiters can directly paste the job description, including skills, qualifications, and experience requirements. This is the primary and most commonly used method.
- **Skill/Requirement Input from Dataset:** Recruiters can select predefined job roles from a dataset dropdown. Once selected, the job title and description are automatically filled, enabling quick and standardized screening.

### 3.3.6 Resume Processing Interface

The processing interface handles execution of the AI screening pipeline:

- **Start Screening Button:** A prominent full-width button labeled “Start Screening” triggers the entire AI processing workflow.
- **Loading Indicator:** A progress bar displays real-time processing status, showing stages such as text extraction, filtering, scoring, and ranking.
- **Processing Status Messages:** Live status updates inform the recruiter about current operations like “Extracting resumes”, “Calculating scores”, and “Ranking candidates”, ensuring transparency during execution.

### 3.3.7 Candidate Ranking Dashboard

The ranking dashboard presents final screening results in an organized format:

- **Ranked Candidate List:** Candidates are displayed in descending order of relevance score, with labels such as Highly Relevant, Relevant, and Not Relevant.
- **Match Score Display:** Each candidate shows an overall percentage score along with detailed breakdowns for skills, experience, education, and semantic similarity.
- **Sorting and Filtering:** Users can filter candidates based on score ranges or search by filename and skills to refine results.
- **Shortlisted Candidates:** Candidates above the threshold are automatically grouped as shortlisted and highlighted for easy selection.

### 3.3.8 Resume Analysis Interface

This section provides detailed insights into individual candidate profiles:

- **Extracted Skills:** Displays matched skills (blue tags) and missing skills (red tags) for quick comparison with job requirements.
- **Experience Details:** Shows experience score based on years, role level, and job relevance.
- **Education Details:** Evaluates qualification matching using a structured degree hierarchy system.
- **Matching Score Breakdown:** Presents detailed scores for skills, experience, education, and semantic similarity, along with an expandable view of extracted resume text.

### 3.3.9 Results Visualization Interface

The analytics section presents screening results using interactive charts:

- **Bar Charts of Scores:** Displays distribution of candidates across relevance categories and most frequently matched skills.
- **Candidate Comparison Graphs:** Includes funnel charts showing filtering stages and pie charts representing score distribution.
- **Skill Matching Visualization:** Shows top skills and experience distribution across candidates using interactive graphs with hover details.

### 3.3.10 Export and Reporting Interface

This section enables exporting and sharing of screening results:

- **CSV/JSON/Excel Export:** Supports multiple formats for downloading structured candidate data.
- **Report Generation:** Automatically generates organized reports with timestamps to avoid overwriting.
- **Shortlisted Export:** Allows exporting only selected shortlisted candidates for HR review.

### 3.3.11 Error Handling and Feedback

The system includes robust validation and user feedback mechanisms:

- **Invalid File Warnings:** Rejects unsupported or corrupted files and displays clear error messages.
- **Missing Input Alerts:** Shows warnings if job description or resumes are not provided before processing.

- **Success Messages:** Confirms successful operations such as upload completion and export generation.

### 3.3.12 User Experience Considerations

The system is designed to ensure smooth and efficient user interaction:

- **Ease of Use:** The workflow is simple and can be completed with minimal steps and no technical knowledge.
- **Consistency:** Uniform design elements such as colors, buttons, and layouts are maintained throughout the system.
- **Readability:** Clean layout, proper spacing, and clear text hierarchy improve usability.
- **Performance Feedback:** Real-time progress indicators keep users informed during processing.
- **Tooltips and Guidance:** Inline help messages assist users in understanding configuration options.

## 3.4 Methodology

### 3.4.1 NLP-Based Feature Extraction Methodology

The feature extraction methodology follows a pipeline approach:

- **Step 1: Short Form Expansion** — A JSON dictionary maps abbreviations (e.g., "JS" → "JavaScript", "ML" → "Machine Learning") and is applied to the raw text using a single compiled regex pattern for efficiency.
- **Step 2: Contact Information Extraction** — Regex patterns extract email addresses, phone numbers, LinkedIn URLs, and GitHub URLs.
- **Step 3: Skills Extraction** — A curated dictionary of 150+ technical skills and 25+ soft skills is matched against the text using word-boundary-aware regex patterns (\b anchors).
- **Step 4: Experience Extraction** — Multiple regex patterns capture experience mentions (e.g., "5+ years of experience", "experience: 5 years", "5 years in software development"). The maximum value is used as the candidate's total experience.
- **Step 5: Education Extraction** — Degree patterns (PhD, Masters, Bachelors, Associates) and field-of-study terms are matched using regex.

### 3.4.2 Matching Methodology

The matching methodology evaluates four dimensions:

- **Skills (40% weight):** Fuzzy matching using RapidFuzz fuzz.ratio() with a threshold of 80% similarity. The score is computed as:

$$\text{skills\_score} = \text{required\_score} \times 0.5 + \text{preferred\_score} \times 0.3 + \text{coverage} \times 0.2$$

where:

$$\text{required\_score} = |\text{matched\_required}| / |\text{required\_skills}|$$

$$\text{preferred\_score} = |\text{matched\_preferred}| / |\text{preferred\_skills}|$$
$$\text{coverage} = |\text{all\_matched}| / |\text{all\_job\_skills}|$$

- **Experience (25% weight):** Computed as the mean of three sub-scores: years match (against the required range), level match (entry/mid/senior/expert), and title relevance (Jaccard similarity between resume titles and job title).
- **Education (15% weight):** Uses a 4-tier degree hierarchy (PhD=4, Masters=3, Bachelors=2, Associates=1) to compare resume degree level against the required degree level.
- **Semantic Similarity (20% weight):** A weighted combination of five similarity metrics computed between the full resume text and the full job description text.

### 3.4.3 Machine Learning Methodology

The ensemble ML pipeline follows a train-predict-blend workflow:

- **Feature Engineering:** 12 numerical features are extracted from each resume-job pair (skills ratios, experience metrics, education levels, semantic scores, text statistics).
- **Model Training:** Three models are trained on labelled data:

XGBoost Regressor (100 estimators, max\_depth=5, lr=0.1)

Random Forest Regressor (100 estimators, max\_depth=10)

Logistic Regression (max\_iter=1000, C=1.0) — trained on binarized labels

Feature Scaling: StandardScaler normalizes features before prediction.

- **Ensemble Prediction:** Weighted average of model predictions:

$$\text{ensemble\_score} = \text{XGB\_pred} \times 0.40 + \text{RF\_pred} \times 0.35 + \text{LR\_prob} \times 0.25$$

- **Optional Stacking:** A meta-model (Random Forest, 50 estimators) is trained on base model predictions for improved ensemble performance.
- **Blending with Rule-Based:** When ML models are available, the final score blends 60% rule-based with 40% ML:

$$\text{total\_score} = \text{rule\_based\_score} \times 0.60 + \text{ml\_score} \times 0.40$$

## CHAPTER 4

### IMPLEMENTATION AND TESTING

#### 4.1 Introduction to Languages, IDE's, Tools and Technologies used for Project work

The Hirix AI Resume Screening System has been developed using a combination of modern programming languages, development tools, machine learning frameworks, and Natural Language Processing (NLP) technologies. These technologies work together to enable automated resume parsing, feature extraction, candidate evaluation, and intelligent ranking based on job requirements.

##### 4.1.1 Programming Languages

###### I. Python

- **Role:** Core language for the entire system — backend processing pipeline, NLP engine, ML models, and web frontend.
- **Features:** Dynamic typing, extensive standard library, support for object-oriented and functional programming, and a rich ecosystem of AI/ML packages via PyPI. Supports asynchronous patterns and clean syntax for rapid prototyping.
- **Application:** Implements the complete two-stage screening pipeline — text extraction, NLP-based feature extraction, regex-driven skill/experience/education parsing, multi-dimensional matching engine, ensemble ML classification, ranking with relevance classification, and the Streamlit web dashboard.

###### II. HTML/CSS

- **Role:** Structure and styling for the Streamlit web interface.
- **Features:** Custom CSS injected via Streamlit's `st.markdown()` for card layouts, skill badges, colour-coded relevance labels, and progress bar styling.

- **Application:** Defines the dashboard layout, resume cards with score breakdowns, matched/missing skill badges (blue and red pill-shaped tags), metric containers, and responsive column grids.

### III. LaTeX

- **Role:** Markup language for project documentation and report generation.
- **Features:** Professional typesetting for equations, algorithms, TikZ diagrams, and structured academic documents.
- **Application:** Used for generating the GNDEC B.Tech project report with UML diagrams, pseudocode listings, mathematical formulae, and formatted tables.

#### 4.1.2 Integrated Development Environment (IDE)

##### 1. Visual Studio Code (VS Code)

- **Role:** Primary IDE for coding, debugging, and testing.
- **Features:** IntelliSense for Python, integrated terminal, Git integration, Python debugger, and extensions such as Pylint, Python, Jupyter, and Prettier.
- **Application:** Used to develop all Python modules (core pipeline, UI, training scripts), run the Streamlit dev server, debug feature extraction logic, and manage Git operations.

#### 4.1.3 Tools and Technologies

##### 1. Streamlit

- **Role:** Web framework for building the interactive recruiter dashboard.
- **Features:** Python-native, reactive execution model, built-in widgets (sliders, file uploaders, tabs, progress bars), session state management, and wide layout support.

- **Application:** Renders the complete screening UI — job description input, resume upload, configuration sidebar, processing controls, results dashboard with five tabbed views, and export functionality.

## 2. PyMuPDF (fitz)

- **Role:** Primary PDF text extraction engine.
- **Features:** Fast page-level text extraction, metadata reading, and image rendering for OCR fallback.
- **Application:** Opens resume PDFs, extracts text page-by-page, and renders pages as images when text extraction yields insufficient content.

## 3. pytesseract + Pillow

- **Role:** OCR (Optical Character Recognition) fallback for scanned PDFs.
- **Features:** Google's Tesseract engine wrapped in Python; Pillow handles image format conversion and preprocessing.
- **Application:** Converts PDF pages to 300 DPI images and runs OCR to extract text from scanned or image-based resumes where PyMuPDF cannot extract text directly.

## 4. NLTK

- **Role:** Natural Language Processing toolkit for text preprocessing.
- **Features:** Tokenization, stopword lists, lemmatization (WordNetLemmatizer), sentence splitting, and corpus utilities.
- **Application:** Tokenizes resume and JD text, removes stopwords, and lemmatizes words for improved keyword matching and TF-IDF vectorization.

## 5. spaCy

- **Role:** Industrial-strength NLP library for Named Entity Recognition.
- **Features:** Pre-trained language models (en\_core\_web\_sm), entity recognition, POS tagging, and dependency parsing.
- **Application:** Extracts named entities (person names, organizations, locations) from resumes to support structured information extraction.

## 6. scikit-learn

- **Role:** Machine learning framework for classification and text vectorization.
- **Features:** TF-IDF vectorizer, Random Forest classifier, Logistic Regression classifier, train/test split, and evaluation metrics (accuracy, precision, recall, F1).
- **Application:** Computes TF-IDF cosine similarity between resumes and JDs; trains Random Forest and Logistic Regression models as ensemble members for relevance prediction.

## 7. XGBoost

- **Role:** Gradient boosting framework for high-accuracy classification.
- **Features:** Optimized gradient-boosted decision trees, regularization, feature importance, and fast training.
- **Application:** Serves as the primary ensemble member (40% weight) in the ML-based relevance prediction pipeline using a 12-dimensional feature vector.

## 8. sentence-transformers

- **Role:** BERT-based semantic similarity computation.
- **Features:** Pre-trained transformer models (all-MiniLM-L6-v2) for generating dense sentence embeddings.
- **Application:** Encodes resume and JD text into vector embeddings and computes cosine similarity for the semantic matching dimension.

## 9. RapidFuzz

- **Role:** Fast fuzzy string matching library.
- **Features:** Levenshtein distance, partial ratio, and token-based matching with C++ backend for speed.
- **Application:** Handles spelling variations, abbreviations, and minor typos in skill names during resume-JD matching.

## 10. pandas

- **Role:** Data manipulation and analysis library.
- **Features:** DataFrames, CSV/Excel reading and writing, data filtering, grouping, and aggregation.
- **Application:** Organizes screening results into DataFrames, reads job postings from postings.csv, and generates structured export files.

## 11. NumPy

- **Role:** Numerical computing library.
- **Features:** N-dimensional arrays, mathematical functions, linear algebra, and vectorized operations.

- **Application:** Performs array operations in the scoring engine, computes weighted averages, and handles matrix calculations for TF-IDF similarity.

## 12. Plotly

- **Role:** Interactive data visualization library.
- **Features:** Bar charts, pie charts, funnel charts, hover tooltips, zoom/pan controls, and PNG export.
- **Application:** Generates all analytics visualizations in the dashboard — relevance distribution, score distribution, pipeline funnel, top skills, category distribution, and experience level charts.

## 13. joblib

- **Role:** Model serialization and persistence.
- **Features:** Efficient serialization of Python objects, particularly large NumPy arrays and scikit-learn models.
- **Application:** Saves trained ensemble models (XGBoost, Random Forest, Logistic Regression) as .pkl files and loads them during screening for inference.

## 14. openpyxl

- **Role:** Excel file generation library.
- **Features:** Reading and writing .xlsx files with formatting support.
- **Application:** Enables export of screening results in Excel format for recruiter convenience.

#### 4.1.4 Developer Productivity and Version Control

1. **Git 2.40+:** Distributed version control system for tracking source code changes, branching, and maintaining project history.
2. **pip + requirements.txt:** Python package manager for installing and managing project dependencies.
3. **dotenv / config.py:** Configuration management for file paths, thresholds, model parameters, and feature flags.

#### 4.1.5 Database (Future Integration)

1. **MySQL 8.0+:** Relational database management system designed for future integration to provide persistent storage of user accounts, job postings, screening results, and candidate records with SQL-based querying and reporting.

#### 4.1.6 Integration with the Resume Screening System

1. **Frontend (Streamlit):** Renders the recruiter dashboard with input forms, configuration controls, processing indicators, and result views. Custom CSS styling provides professional card layouts, colour-coded badges, and responsive column grids.
2. **Core Pipeline (Python):** The processing engine orchestrates the two-stage pipeline — Stage 1 coarse filtering on raw text, followed by Stage 2 NLP feature extraction, multi-dimensional matching, ML ensemble scoring, and relevance classification.
3. **NLP Layer (NLTK + spaCy):** Preprocesses text through tokenization and lemmatization, then extracts structured features (skills, experience, education, contact info) using regex patterns and NER models.

4. **ML Layer (scikit-learn + XGBoost + sentence-transformers):** Computes TF-IDF and BERT semantic similarity, runs ensemble classification, and blends rule-based and ML scores into a final relevance score.
5. **Data Layer (pandas + File System):** Reads resume PDFs and job postings, organizes results into DataFrames, and exports to CSV/Excel/JSON formats. Future MySQL integration will replace file-based storage.
6. **Development Workflow:** VS Code drives day-to-day development with integrated terminal and Python debugging. Git manages source control. Configuration is centralized in config.py with thresholds, weights, and file paths as constants.

## 4.2 Algorithm/Pseudocode

The Hirix AI Resume Screening System employs efficient algorithms to manage its core resume processing, feature extraction, and candidate ranking functionality. Three key algorithms are outlined: the Complete Resume Screening Pipeline, TF-IDF Cosine Similarity, and Fuzzy Skills Matching. These algorithms handle the end-to-end processing of resumes from raw PDF text extraction to final AI-driven classification. Implemented primarily in the backend core modules (`preprocessor.py`, `feature\_extractor.py`, `ranker.py`), these algorithms ensure accurate, high-performance, and scalable resume screening. Below, each algorithm is briefly explained, focusing on its purpose, steps, and role in the Hirix system.

### 4.2.1 Complete Resume Screening Pipeline

- **Purpose:** This is the master algorithm that orchestrates the entire two-stage screening process. It handles extracting text from PDF resumes, parsing the job description, applying a fast coarse filter (Stage 1), extracting detailed NLP features (Stage 2), and computing the final relevance scores using both rule-based heuristics and machine learning models.
- **Pseudocode:**

```
```text
```

```
Function ScreenResumes(job_text, resume_files, config):
```

```
    START_TIMER()
```

```
    // STEP 1: Parse Job Description
```

```
    job_reqs = JDProcessor.process(job_text)
```

```

// STEP 2: Extract Text from PDFs (with OCR fallback)

extracted = EMPTY_MAP

For Each pdf IN resume_files:

text = extract_text_with_pymupdf(pdf)

if length(text) < 100:

text = extract_text_with_tesseract_ocr(pdf)

extracted[pdf] = clean_text(text)

// STEP 3: Stage 1 - Coarse Filter

passed = EMPTY_MAP

if config.coarse_filter_enabled:

For Each (path, text) IN extracted:

coarse_score = compute_overlap_and_coverage(text, job_reqs)

if coarse_score >= 0.15:

passed[path] = text

else:

passed = extracted

// STEP 4: Stage 2 - Feature Extraction

features = EMPTY_MAP

For Each (path, text) IN passed:

```

```

f = NEW ResumeFeatures()

f.technical_skills = MATCH_DICT(tech_skills_dict, text)

f.experience_years = MAX(REGEX_FIND_ALL(exp_patterns, text))

f.degrees = REGEX_FIND_ALL(degree_patterns, text)

features[path] = f

// STEP 5: Matching and Scoring

scores = EMPTY_MAP

For Each (path, f) IN features:

    skill_score = fuzzy_match_skills(f.skills, job_reqs.required)

    exp_score = match_experience(f.years, job_reqs.min, job_reqs.max)

    sem_score = compute_semantic(f.raw_text, job_reqs.raw_text)

    total = (skill * 0.40) + (exp * 0.25) + (sem * 0.20) + (edu * 0.15)

    if config.ml_enabled:

        ml_score = ensemble.predict(prepare_features(f, job_reqs))

        total = (total * 0.60) + (ml_score * 0.40)

    scores[path] = total

// STEP 6: Ranking and Classification

ranked = SORT(scores, BY total, DESCENDING)

For Each result IN ranked:

```

```
assign_label_based_on_threshold(result)
```

```
STOP_TIMER()
```

```
return ranked_results
```

```
...
```

#### 4.2.2 TF-IDF Cosine Similarity

- **Purpose:** This algorithm calculates the semantic similarity between the unstructured text of a candidate's resume and the target job description. By converting text into Term Frequency-Inverse Document Frequency (TF-IDF) vectors and measuring the cosine of the angle between them, the system can determine how closely the context and vocabulary of the resume align with the job requirements.
- **Pseudocode:**

```
```text
```

```
Function TF_IDF_Cosine(resume_text, jd_text):
```

```
// Step 1: Initialize Vectorizer
```

```
vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1,2))
```

```
// Step 2: Compute TF-IDF matrix for both documents
```

```
matrix = vectorizer.fit_transform([resume_text, jd_text])
```

```
// matrix is a sparse array containing weighted term frequencies
```

```
// Step 3: Compute cosine similarity
```

```
// Formula:  $\cos(\theta) = (A \cdot B) / (\|A\| \times \|B\|)$ 
```

```

similarity_score = cosine_similarity(matrix[0], matrix[1])

return similarity_score

'''

```

#### 4.2.3 Fuzzy Skills Matching

- **Purpose:** Resumes often contain slight spelling variations, abbreviations, or formatting differences for technical skills (e.g., "React.js" vs "ReactJS"). This algorithm uses Levenshtein distance (via the RapidFuzz library) to perform fuzzy string matching, ensuring that candidates are not unfairly penalized for minor typographical differences when comparing their extracted skills against the required job skills.
- **Pseudocode:**

```

'''text

Function FuzzySkillMatch(required_skill, extracted_resume_skills, raw_resume_text,
threshold=80):

target_skill = LOWERCASE(required_skill)

// Phase 1: Fuzzy match against structured extracted skills

For Each candidate_skill IN extracted_resume_skills:

similarity = LEVENSHTEIN_RATIO(target_skill, candidate_skill)

if similarity >= threshold:

return TRUE

```

```
// Phase 2: Fallback to exact substring match in raw text
```

```
if      REGEX_SEARCH("\b"      +      ESCAPE(target_skill)      +      "\b",
```

```
LOWERCASE(raw_resume_text)):
```

```
    return TRUE
```

```
    return FALSE
```

```
...
```

### **4.3 Testing Techniques**

Testing plays an important role in ensuring the reliability, accuracy, and performance of the Hirix AI Resume Screening System. Various testing techniques were applied to verify the correctness of individual modules, integration between system components, user experience, and overall system performance.

#### **4.3.1 Unit Testing**

**Purpose:**

Unit testing was performed to verify the functionality of individual modules independently.

**Modules Tested:**

- Resume text extraction module
- Feature extraction module
- Candidate ranking module
- Machine learning scoring module
- User interface components

**Testing Activities:**

- Verification of PDF text extraction and OCR functionality.
- Validation of skill, experience, and education extraction.
- Testing of similarity calculations and scoring algorithms.
- Verification of candidate ranking accuracy.

**Outcome:**

All individual modules produced expected outputs and handled invalid inputs appropriately.

#### 4.3.2 Integration Testing

- **Purpose:**

Integration testing was conducted to ensure smooth interaction between different modules of the system.

- **Testing Activities:**

- Integration of resume upload and text extraction modules.
- Integration of NLP feature extraction and matching engine.
- Integration of machine learning models with the ranking system.
- Integration of result visualization and export functionalities.

- **Outcome:**

The system successfully processed resumes from upload to final ranking without data loss or inconsistencies.

#### 4.3.3 Usability Testing

- **Purpose:**

Usability testing was performed to evaluate the ease of use and effectiveness of the recruiter interface.

- **Testing Activities:**

- Entering job descriptions.
- Uploading multiple resumes.
- Running the screening process.
- Viewing candidate rankings and analytics.
- Exporting shortlisted candidates.

- **Outcome:**

The interface was found to be intuitive, easy to navigate, and suitable for recruiters with minimal technical knowledge.

#### **4.3.4 Performance Testing**

- **Purpose:**

Performance testing was conducted to evaluate the system's speed and efficiency while processing large numbers of resumes.

- **Testing Activities:**

- Measuring resume processing time.
- Evaluating response time of the ranking engine.
- Monitoring memory and CPU utilization.
- Testing batch processing of multiple resumes.

- **Outcome:**

The two-stage screening architecture significantly reduced processing time and maintained stable performance under heavy workloads.

#### **4.3.5 Data Integrity and Security Testing**

- **Purpose:**

This testing ensured that the system handled uploaded files securely and maintained data integrity.

- **Testing Activities:**

- Validation of uploaded PDF files.
- Detection of corrupted or unsupported files.
- Verification of input sanitization.

- Secure generation of exported reports.
- **Outcome:**

The system successfully prevented invalid inputs from affecting processing and maintained the integrity of screening results.

#### **4.3.6 Regression and Smoke Testing**

- **Purpose:**

Regression and smoke testing were performed to ensure that newly added features did not affect existing functionality.
- **Testing Activities:**
  - Verification of application startup and initialization.
  - Testing core screening functionality after updates.
  - Validation of scoring and ranking consistency.
  - Verification of export and visualization features.
- **Outcome:**

All critical functionalities operated correctly after modifications, confirming the stability and reliability of the system.

## 4.4 Test Cases

This section outlines the critical test cases executed to verify the functionality, accuracy, and error handling of the Hirix AI Resume Screening System. The test cases are categorized into UI validation, text extraction, NLP processing, scoring mechanisms, and system outputs.

### 4.4.1. Test Cases for Text Extraction Module

*Table 4.1 Test Cases for Text Extraction Module*

| TC ID | Test Scenario                 | Input                    | Expected Output                               |
|-------|-------------------------------|--------------------------|-----------------------------------------------|
| TE-01 | Extract text from digital PDF | Valid 2-page digital PDF | success=True, method='direct', word_count>100 |
| TE-02 | Extract text from scanned PDF | Scanned image-based PDF  | success=True, method='ocr'                    |
| TE-03 | Handle missing file           | Non-existent path        | success=False, error='File not found'         |
| TE-04 | Handle oversized file         | 15 MB PDF                | success=False, error='File too large'         |
| TE-05 | Handle corrupt PDF            | Truncated file           | success=False, error message                  |
| TE-06 | Handle empty PDF              | 0-page PDF               | success=False, error='No text'                |
| TE-07 | Batch extraction              | 10 valid PDFs            | 10 success results                            |

#### 4.4.2 Test Cases for Feature Extraction Module

*Table 4.2 Test Cases for Feature Extraction Module*

| TC ID | Test Scenario           | Input                       | Expected Output           |
|-------|-------------------------|-----------------------------|---------------------------|
| FE-01 | Extract email           | "john@email.com" in resume" | "john@email.com"          |
| FE-02 | Extract phone           | "Phone: (555) 123-4567"     | "(555) 123-4567"          |
| FE-03 | Extract Python skill    | "Expert in Python and Java" | ['java', 'python']        |
| FE-04 | Extract experience      | "5+ years of experience"    | 5.0                       |
| FE-05 | No experience mentioned | "Recent graduate, no exp."  | 0.0                       |
| FE-06 | Extract Masters degree  | "M.S. in Computer Science"  | ['m.s.'] or ['master']    |
| FE-07 | Extract LinkedIn        | "linkedin.com/in/johndoe"   | "linkedin.com/in/johndoe" |
| FE-08 | Empty text input        | ""                          | Empty ResumeFeatures      |

#### 4.4.3 Test Cases for Job Description Processor

*Table 4.3 Test Cases for Job Description Processor*

| TC ID | Test Scenario            | Input                       | Expected Output                       |
|-------|--------------------------|-----------------------------|---------------------------------------|
| JD-01 | Extract required skills  | JD with Python, AWS, ML     | ['aws', 'machine learning', 'python'] |
| JD-02 | Extract experience range | "5-7 years of experience"   | min=5.0, max=7.0                      |
| JD-03 | Detect fresher level     | "0-3 years, entry-level"    | min=0.0, max=3.0                      |
| JD-04 | Extract degree           | "Bachelor's in CS required" | "bachelors"                           |
| JD-05 | Detect remote work       | "Location: Remote / Hybrid" | remote_allowed=True                   |

| TC ID | Test Scenario    | Input                      | Expected Output        |
|-------|------------------|----------------------------|------------------------|
| JD-06 | Extract salary   | "\$150K - \$200K per year" | min=150000, max=200000 |
| JD-07 | Extract job type | "Full-time position"       | "full-time"            |

#### 4.4.4 Test Cases for Matching Engine

*Table 4.4 Test Cases for Matching Engine*

| TC ID | Test Scenario                     | Input                               | Expected Output                    |
|-------|-----------------------------------|-------------------------------------|------------------------------------|
| MA-01 | Perfect skills match              | Resume has all required + preferred | skills_score $\approx$ 1.0         |
| MA-02 | No skills match                   | Resume has none of required skills  | skills_score $\approx$ 0.0         |
| MA-03 | Partial skills match              | 3 of 5 required skills              | $0.3 < \text{skills\_score} < 0.8$ |
| MA-04 | Fuzzy skill match                 | "sklearn" vs "scikit-learn"         | Matched (ratio $\geq$ 80)          |
| MA-05 | Experience within range           | 5yr resume, 3-8yr required          | experience_score = 1.0             |
| MA-06 | Experience below range            | 1yr resume, 5yr required            | experience_score $<$ 0.5           |
| MA-07 | Degree meets requirement          | MS resume, BS required              | education_score = 1.0              |
| MA-08 | Degree below requirement          | BS resume, MS required              | education_score $<$ 1.0            |
| MA-09 | Semantic similarity (same domain) | ML resume + ML JD                   | semantic_score $>$ 0.3             |

| TC ID | Test Scenario                          | Input             | Expected Output       |
|-------|----------------------------------------|-------------------|-----------------------|
| MA-10 | Semantic similarity (different domain) | HR resume + ML JD | semantic_score < 0.15 |

#### 4.4.5 Test Cases for Ranking Module

*Table 4.5 Test Cases for Ranking Module*

| TC ID | Test Scenario               | Input                     | Expected Output                         |
|-------|-----------------------------|---------------------------|-----------------------------------------|
| RK-01 | Rank 10 resumes             | 10 match results          | Sorted DESC by score                    |
| RK-02 | Classify as relevant        | Score = 0.75              | is_relevant=True, label="Relevant"      |
| RK-03 | Classify as not relevant    | Score = 0.25              | is_relevant=False, label="Not Relevant" |
| RK-04 | Rejection reasons generated | Score = 0.20, skills=0.15 | "Low skills match (15%)"                |
| RK-05 | Coarse filter pass          | Good resume text          | should_process=True                     |
| RK-06 | Coarse filter fail          | Empty/short text          | should_process=False                    |
| RK-07 | Export CSV                  | Ranked results            | Valid CSV file created                  |
| RK-08 | Export JSON                 | Ranked results            | Valid JSON file created                 |

## CHAPTER 5

### RESULTS AND DISCUSSIONS

#### 5.1 User Interface Representation

The Hirix AI Resume Screening System provides a simple, interactive, and user-friendly interface that enables recruiters to perform resume screening efficiently. The interface is developed using Streamlit and is designed to simplify the recruitment process by integrating resume analysis, candidate ranking, and result visualization into a single platform.

The system allows recruiters to upload resumes, provide job descriptions, configure screening parameters, and view ranked candidate results through an organized dashboard. Visual elements such as progress indicators, score cards, charts, and status messages enhance usability and improve the overall user experience.

##### a. Screening Configuration Flow

- The recruiter inputs the job requirements by pasting text, uploading a file (.txt, .pdf, .docx), or selecting from an existing dataset.
- The recruiter supplies candidate resumes via bulk PDF upload or by selecting specific pre-categorized dataset folders.
- Through an always-accessible sidebar, the user can fine-tune AI parameters, including relevance thresholds, machine learning model selection, and individual scoring weights (Skills, Experience, Education, Semantic).

## **b. AI Processing and Dashboard Experience**

- Upon initiating the screening, users receive real-time visual feedback via an animated progress bar indicating the current pipeline stage (Extraction → Filtering → Matching → Ranking).
- After processing, users access a comprehensive dashboard displaying summary metrics and categorized candidate lists (Relevant vs. Non-Relevant).
- Recruiters can drill down into individual candidate cards to view detailed score breakdowns, matched/missing skill badges, and interactive Plotly analytics charts.

### **5.1.1 Brief Description of Various Modules of the System**

The Hirix system is divided into multiple interconnected modules across the frontend interface, core processing pipeline, and data management layers, each responsible for specific functionality.

#### **1. Frontend Application (Streamlit)**

The frontend provides the user-facing interface and manages all interaction logic. It communicates seamlessly with the underlying Python processing modules to orchestrate the screening workflow.

##### **a. Configuration and Input Views**

- **Sidebar Module:** Encapsulates logic for system configuration, allowing recruiters to toggle the two-stage pipeline, adjust scoring weights, and enable OCR fallback.
- **Input Handlers:** Manages the uploading and validation of multiple PDF resumes and job description files, ensuring format integrity before passing data to the backend.

- **Feedback Mechanisms:** Displays dynamic error alerts (e.g., missing JD) and informational messages regarding dataset counts.

## **b. Processing and Results Views**

- **Dashboard Component:** Renders the main results interface, organizing candidates into tabbed views for easy navigation.
- **Candidate Cards:** Responsible for displaying individual candidate summaries with proper visual hierarchy:
  - Overall match percentage and rank.
  - Granular progress bars for Skills, Experience, Education, and Semantic similarity.
  - Badge styling (blue for matched skills, red for missing skills).
- **Analytics Component:** Integrates with the Plotly library to render interactive funnel charts, score distribution pie charts, and top-skills bar charts.

## **2. Core Backend Engine (Python NLP/ML Layer)**

The backend serves as the core logic layer, handling data extraction, natural language processing, intelligent scoring, and candidate classification.

### **a. Text Extraction Module**

- **File:** core/text\_extractor.py`
- **Responsibilities:**
  - Extracts raw text from uploaded PDFs using PyMuPDF.

- Automatically detects scanned documents and triggers a Tesseract OCR fallback.
- Cleans and sanitizes extracted text for NLP processing.

#### **b. NLP Feature Extraction Module**

- **File:** core/feature\_extractor.py
- **Responsibilities:**
  - Tokenizes and lemmatizes text using NLTK.
  - Extracts structured entities (technical skills, soft skills, years of experience, degree levels) using regex patterns and spaCy Named Entity Recognition.
  - Parses the job description to identify mandatory vs. preferred requirements.

#### **c. Scoring and Ranking Module**

- **File:** core/ranker.py
- **Responsibilities:**
  - Executes the Stage 1 coarse filter to eliminate highly irrelevant resumes early.
  - Computes TF-IDF cosine similarity and BERT-based semantic embeddings.
  - Generates final predictions using a blended approach of rule-based heuristics and a trained Machine Learning ensemble (XGBoost, Random Forest, Logistic Regression).
  - Assigns relevance labels (Highly Relevant, Relevant, Not Relevant) based on user-defined thresholds.

### 3. Data Management and Export Layer

- **In-Memory Storage:** Utilizes pandas DataFrames to efficiently store, sort, and filter candidate data during an active session.
- **Export Module:** Maps internal DataFrame structures to downloadable formats, generating timestamped CSV, Excel (.xlsx), and JSON reports for both combined and classified results.
- **Future Integration (MySQL):** The system architecture is designed to support a planned transition to a MySQL relational database for persistent storage of job postings, candidate records, and historical screening metrics.

### 4. Integration and User Experience

The system integrates file parsing, AI inference, and data visualization into a cohesive, single-page application.

- Expandable sections (e.g., "View Resume Text", "Failed Extractions") ensure transparency without cluttering the UI.
- Exception handling and input validation boundaries ensure reliability and smooth recovery from corrupted files or missing data.
- Responsive container layouts ensure the dashboard remains accessible and readable on varying screen sizes and resolutions.

## 5.2 Snapshots of system with brief detail of each and discussion

This section presents the major interfaces of the Hirix AI Resume Screening System along with their functionality and significance in the resume screening process.

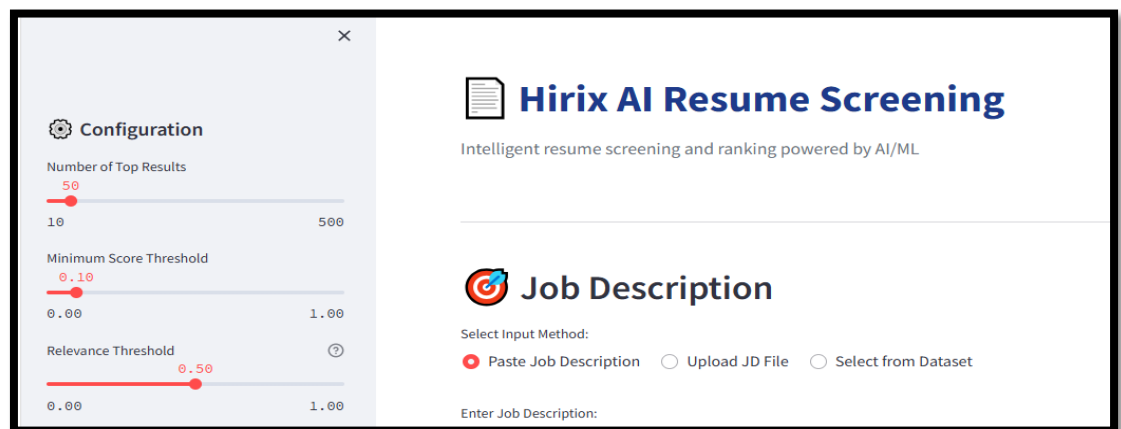
### 5.2.1 Configuration and Input Screens

- **Home Page & Sidebar Configuration**

The home page serves as the entry point of the system. It contains the project title, system description, and a sidebar with configuration settings such as relevance threshold, scoring weights, and pipeline options.

- **Discussion:**

The sidebar allows recruiters to customize screening parameters according to job requirements. Its organized layout improves usability and simplifies system configuration.



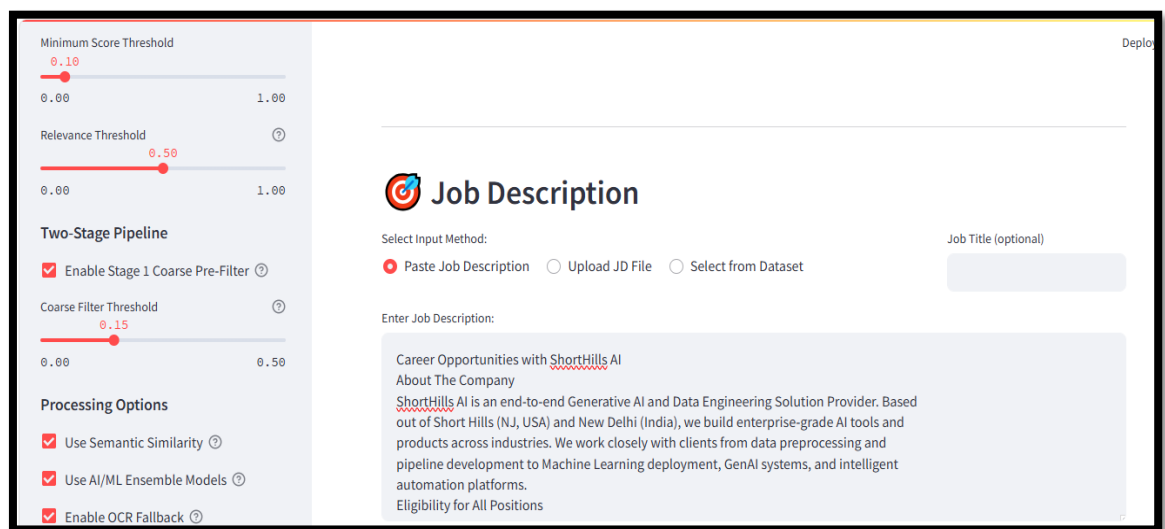
***Fig.5.1 Home Page & Sidebar Configuration***

- **Job Description Input Interface**

This screen allows recruiters to provide job descriptions either by entering text manually, uploading a file, or selecting a predefined job posting from the dataset.

- **Discussion:**

Multiple input methods provide flexibility and reduce manual effort. Recruiters can quickly define job requirements for candidate evaluation.



*Fig.5.2 Job Description Input Interface*

- **Resume Upload Interface**

The resume upload screen supports uploading one or multiple PDF resumes. Users can also select resumes from predefined dataset categories.

- **Discussion:**

The interface simplifies bulk resume submission and ensures that uploaded files are validated before processing.



**Fig.5.3 Resume Upload Interface**

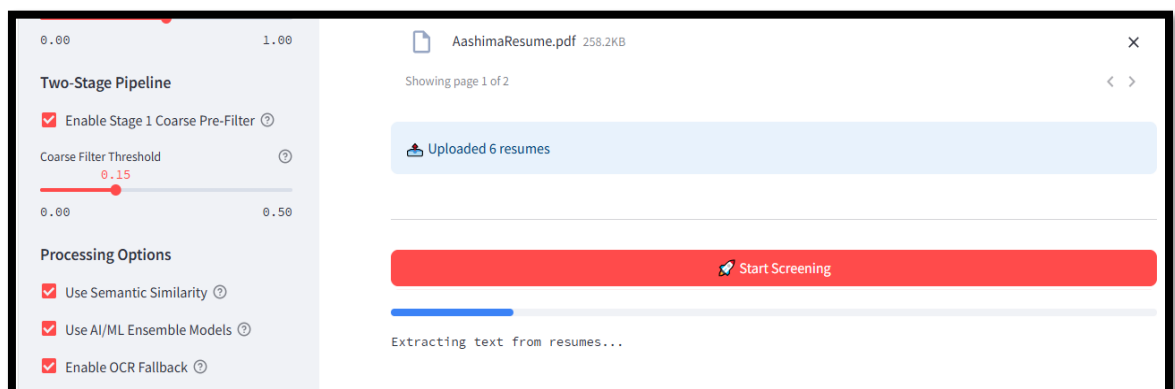
## 5.2.2 Processing and Real-Time Feedback

- **AI Processing Screen**

After clicking the “Start Screening” button, the system begins processing resumes and displays a progress bar along with status updates.

- **Discussion:**

Real-time feedback keeps users informed about the current processing stage and improves the overall user experience.



**Fig.5.4 Processing and Real-Time Feedback**

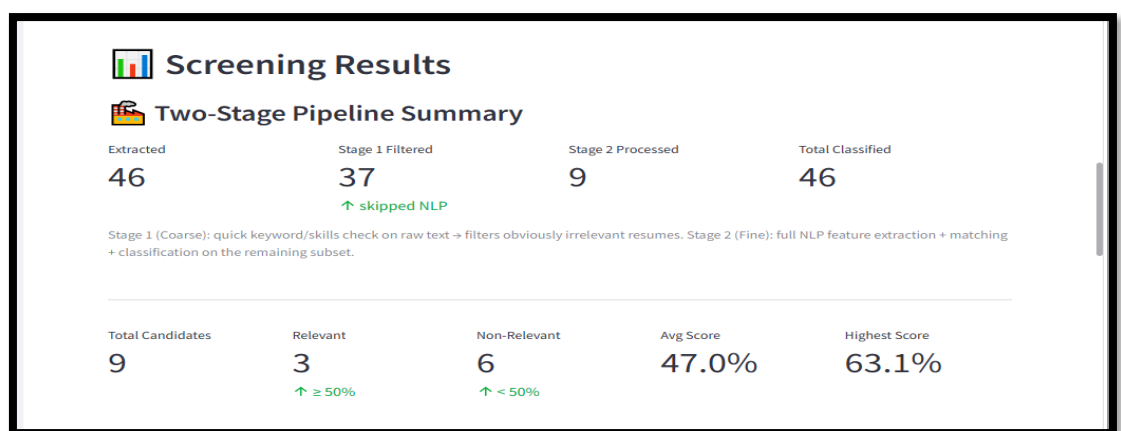
- **Two-Stage Pipeline Summary Banner**

The interface includes a dedicated toggle in the sidebar to enable the Two-Stage Pipeline architecture. When enabled, a summary banner appears at the top of the results dashboard displaying key metrics, including:

- Total Resumes Extracted
- Candidates Filtered Out in Stage 1
- Candidates Processed in Stage 2
- Final Classified Count

- **Discussion:**

This screen demonstrates the performance optimization achieved by the two-stage pipeline. The Stage 1 coarse filter quickly removes highly irrelevant resumes before detailed NLP and machine learning processing is performed. By processing only the shortlisted resumes in Stage 2, the system reduces computational overhead, shortens screening time, and improves overall efficiency while maintaining result quality.



**Fig.5.5 Two-Stage Pipeline Summary Banner**

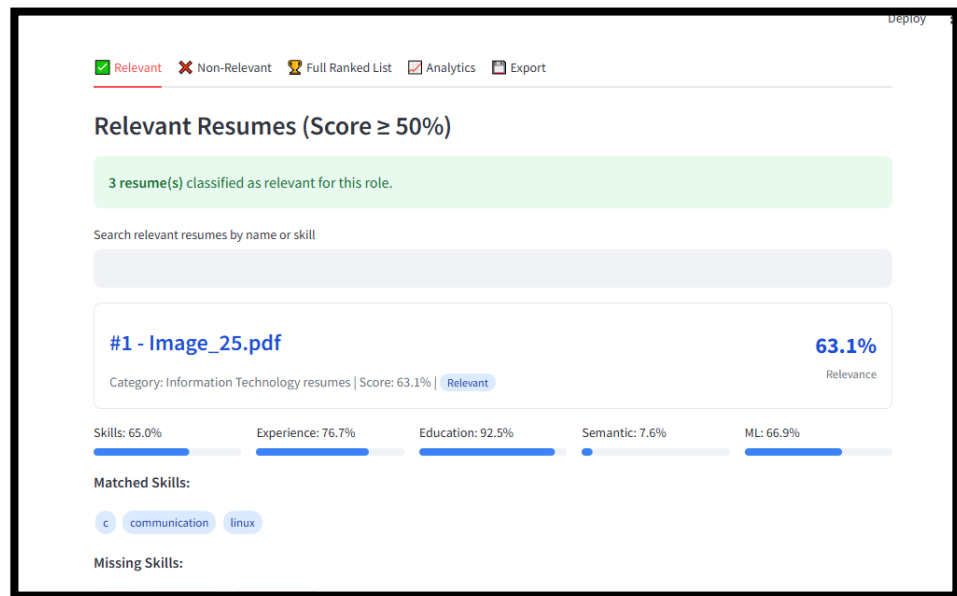
### 5.2.3 Candidate Evaluation Dashboards

- **Candidate Ranking Dashboard**

This dashboard displays all screened candidates ranked according to their relevance scores. Candidates are categorized as Highly Relevant, Relevant, or Not Relevant.

- **Discussion:**

The ranking dashboard helps recruiters quickly identify the most suitable candidates and reduces manual screening effort.



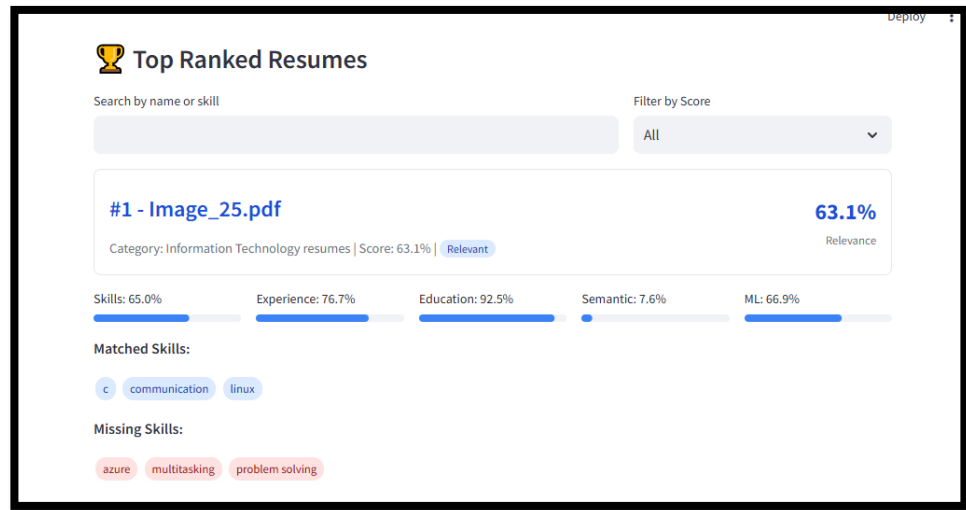
*Fig.5.6 Candidate Ranking Dashboard*

- **Resume Analysis Interface**

This screen provides detailed information about an individual candidate, including skills, experience, education details, and score breakdown.

- **Discussion:**

The detailed analysis improves transparency by showing how the final score was calculated and highlighting candidate strengths and skill gaps.



***Fig.5.7 Resume Analysis Interface***

## CHAPTER 6

### CONCLUSION AND FUTURE SCOPE

#### 6.1 Conclusion

The Hirix AI Resume Screening System successfully demonstrates the application of Artificial Intelligence and Natural Language Processing (NLP) in automating and optimizing the recruitment workflow. By replacing the tedious, error-prone process of manual resume screening with a fast, intelligent, and data-driven pipeline, the system significantly reduces the time-to-hire for HR professionals.

The implementation of a two-stage screening architecture ensures high computational efficiency. The Stage 1 coarse filter rapidly eliminates highly irrelevant resumes using keyword overlap heuristics, reserving the intensive NLP extraction and Machine Learning (ML) predictions for viable candidates in Stage 2. The integration of robust text extraction tools (PyMuPDF and Tesseract OCR) ensures that both digital and scanned resumes are accurately parsed.

Furthermore, the multi-dimensional scoring engine—evaluating technical/soft skills, years of experience, educational degrees, and TF-IDF/BERT semantic similarity—provides a holistic evaluation of a candidate's fit for a specific job description. The blending of rule-based logic with an ensemble ML model (XGBoost, Random Forest, Logistic Regression) ensures scoring accuracy. Finally, the development of a clean, responsive web dashboard using Streamlit successfully abstracts this technical complexity, providing recruiters with an intuitive, transparent, and visually rich tool to rank candidates, analyze skill gaps, and export actionable hiring reports.

## 6.2 Future Scope

While the current implementation of the Hirix system is highly functional and accurate, several enhancements can be made to scale the application and broaden its capabilities in future iterations:

- **Persistent Database Integration:** Currently, the system operates largely in-memory using pandas DataFrames and file-based storage. Future versions will integrate a relational database management system (such as MySQL or PostgreSQL) to provide persistent storage for user accounts, historical job postings, saved candidate pools, and long-term recruitment analytics.
- **Advanced Large Language Model (LLM) Integration:** While the system currently uses BERT embeddings for semantic similarity, integrating advanced generative models (such as OpenAI's GPT-4 or Meta's LLaMA) could allow the system to generate human-readable, paragraph-length summaries of a candidate's strengths and weaknesses, further aiding recruiter decision-making.
- **Applicant Tracking System (ATS) Integration:** Developing secure RESTful APIs (using frameworks like FastAPI) will allow Hirix to plug directly into enterprise Applicant Tracking Systems (e.g., Workday, Greenhouse, or Lever). This would allow resumes submitted on a company website to be routed directly into Hirix for real-time scoring.
- **Bias Mitigation and Fairness Auditing:** As with any ML system used in hiring, there is a risk of inheriting historical biases. Future work will include implementing algorithmic fairness checks to ensure the ML ensemble models do not unintentionally penalize candidates based on demographic indicators inferred from their resumes.

- **Enhanced Multi-Lingual and Format Support:** Extending the NLP engine to process resumes written in multiple languages and improving native support for parsing complex Word documents (.docx) with embedded tables and varied formatting structures.

## References

- [1] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA, USA: O'Reilly Media, 2009.
- [2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.
- [3] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.
- [4] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Hong Kong, China, 2019, pp. 3982–3992.
- [6] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [7] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] A. Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

- [9] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, Minneapolis, MN, USA, 2019, pp. 4171–4186.
- [10] M. Honnibal and I. Montani, “spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing,” 2017. [Online]. Available: <https://spacy.io>
- [11] Streamlit Inc., “Streamlit – The fastest way to build data applications.” [Online]. Available: <https://streamlit.io>
- [12] R. Smith, “An overview of the Tesseract OCR engine,” in *Ninth International Conference on Document Analysis and Recognition (ICDAR)*, vol. 2, 2007, pp. 629–633.
- [13] PyMuPDF Documentation, “PyMuPDF (fitz) – Python bindings for MuPDF.” [Online]. Available: <https://pymupdf.readthedocs.io>
- [14] NLTK Project, “Natural Language Toolkit (NLTK) Documentation.” [Online]. Available: <https://www.nltk.org>
- [15] RapidFuzz Developers, “RapidFuzz Documentation.” [Online]. Available: <https://rapidfuzz.github.io>
- [16] Plotly Technologies Inc., “Plotly Python Graphing Library Documentation.” [Online]. Available: <https://plotly.com/python>
- [17] Python Software Foundation, “Python 3 Documentation.” [Online]. Available: <https://docs.python.org/3/>